



M363 Unit 3
UNDERGRADUATE COMPUTING

Software engineering with objects



From domain modelling
to requirements
analysis

Unit **3**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1609 2



CONTENTS

1	Introduction	5
2	Business rules and business processes	6
	2.1 Business rules	6
	2.2 Business processes	8
	2.3 Summary of section	18
3	Simple use case models	19
	3.1 Use cases	19
	3.2 Emphasis on the user	20
	3.3 Stepping from the problem to its solution	22
	3.4 Actors	23
	3.5 Describing use cases	24
	3.6 Use cases in development activities	28
	3.7 Summary of section	34
4	More details on use case models	35
	4.1 More about actors	35
	4.2 Modelling the relationships between use cases	36
	4.3 Problems with use cases	43
	4.4 Summary of section	49
5	Getting users involved	51
	5.1 Prototyping for and with the user	51
	5.2 Linking activities to user interfaces	53
	5.3 Summary of section	57
6	Summary	58
	References	59
	Index	60

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we study some well established techniques for capturing and documenting user requirements and system behaviour.

First, we study activity diagrams, which model workflows. Activity diagrams can be used to model business processes in a domain, and we could, if we wished, use them quite independently of software development.

The main concepts in a business domain are its *resources* (people, products, information and materials), *processes* (the activities that manipulate, transform and use these resources), *rules* that constrain the processes, and *goals* that define the purpose of the business (Eriksson and Penker, 2001).

Understanding the business is an essential step in developing any software solution, as the business is the context in which the software system will be applied, and software based on an inadequate analysis is almost guaranteed to fail. Section 2 introduces the concepts of business rules and business processes; activity diagrams are introduced as the technique for modelling domain processes.

Next, we look at use cases and use case diagrams, which record chunks of functionality that users want from a system, and study how these relate to one another, how they represent interaction with the users, and how they can be documented precisely. In Sections 3 and 4, we assume a decision has been taken to develop a software system to solve a specific problem within a particular business, and the initial set of user requirements are the basis for what will be modelled.

Finally, in Section 5, we consider how prototyping can help with requirements engineering. At the end, we bring all these ideas together and consider how activity diagrams can help us identify where user interfaces will be needed.

2

Business rules and business processes

On completion of this section you should be able to:

- ▶ explain the concepts of business rules and business processes;
- ▶ identify business rules from problem descriptions;
- ▶ represent processes and rules in models of the business;
- ▶ draw an activity diagram from a simple problem description.

2.1 Business rules

Business rules constrain how a business is run. Understanding the business rules is key to modelling the business and to specifying how any system to be developed can support the correct functioning of the business processes. There is a clear distinction between the business processes and the constraints imposed on them. For example, in a car rental company, renting a car is one of the processes of the business; one business rule that may apply to this process is that the car allocated is the lowest-mileage car that is available in the chosen group.

Example 1

Consider the following extract from a business description.

A typical lending library keeps a stock of books for the use of its members. Each member can take out a number of books, up to a certain limit. After a given period of time, the library expects members to return the books that they have on loan.

When borrowing books, a member is expected to hand their chosen books to the librarian, who records each new loan before issuing the books to the member. When a book is on loan to a member, it is associated with that member: possession of the book passes from the library to the member for a defined period. The normal loan period for each book is two weeks. If the member fails to bring the book back on or before the due date, the library imposes a fine.

We can identify two business processes in this domain: issue a book, and return a book.

The following business rules characterise the way the business operates.

- ▶ There is a limit to the number of books a member can take out.
- ▶ A loan is for a period that is normally two weeks.
- ▶ Late returns incur a fine.

Representing business rules

It is important that business rules are identified and recorded, as they are the basis of decision making. They are also the part of the business that is more likely to change as the business adapts to external pressures, to become more competitive or to seize new opportunities. Ideally, a record of the business rules should be kept, to provide quick and easy access whenever there is a change.

Business rules need to be correctly represented, and they should support the business processes and strategy. Ideally, business rules should be represented independently of other models, to facilitate analysis. Any reasonable representation should support their verification and validation before any system is built. **Verification** is the set of activities that ensure that the rules are correctly modelled; **validation** ensures that they are the most appropriate rules for the business. Verification can be done automatically if the rules are well specified; the models in development should be checked to comply with these rules. Validation of the rules is not an easy process, and can only be done through review by the stakeholders; are the rules the ones intended for what the business wants to achieve? There is an argument for basing the rules on natural language, as they need to be read and understood by business people; in this case, the language used should be well defined and structured, so that verification can be made easier if not automatic.

UML and OCL can also be used to represent business rules. Some of the rules identified for the lending library can be represented in a UML model either as annotations or using the features of the model; we will look later at how each model represents the constraints imposed. UML lacks the full power needed for the representation of business rules. Business rules are not first class citizens in UML: there is no UML notation to model them explicitly and independently; they always appear as part of other models and are therefore more difficult to manage. Ideally, you should be able to analyse and validate business rules and change them as the business evolves. Traceability should be established between the rules of the business and their representation throughout the development process and implementation.

Object Constraint Language (OCL) is a formal language used to represent constraints in UML models.

SAQ 1

- (a) What are the important properties that a representation of business rules should have?
- (b) Consider whether business rules can be modelled in UML; discuss the consequences in the light of the properties in your answer to (a).

ANSWER.....

- (a) The important properties are as follows:
 - ▶ business rules should be represented separately from other models;
 - ▶ they should be easy to verify (possibly automatically) and validate;
 - ▶ they should be represented in a readable but structured language.
 - (b) UML does not provide an explicit notation for business rules. As a consequence, it does not:
 - ▶ facilitate separate recording of the rules;
 - ▶ facilitate their analysis, validation and change;
 - ▶ facilitate their traceability from the business needs to the software solution.
-

2.2 Business processes

Business processes define what is done in a business, by whom, in what order, needing which resources, and with what consequences. Much has been written on process modelling, and there are many techniques to do it. A full presentation of such techniques is, however, well outside the scope of this course. The main reason we look at business processes is to get an understanding of what a business does, and to gain the domain expertise needed to develop software solutions in the business context.

Representing business processes

Workflow is defined as 'a sequence of activities that produces a result of observable value' (Krutchen, 2000). Here we discuss how to represent these activities.

In this course we model business processes using UML **activity diagrams**. An activity diagram shows a process as a set of **activities**, and describes how they must be coordinated – which ones depend on others having been completed first, and where activities can be carried out in parallel.

Activity diagrams help us to understand the business situation before any decisions are taken about a software solution and its boundary. They model behavioural aspects of the domain.

The notion of an activity is very much dependent on the context in question. At the conceptual level, an activity could be a task that a person or a computer might perform. In this section, the emphasis is on the use of activity diagrams to support requirements engineering. When modelling requirements, developers try to understand the problem domain. They want to know what their users are trying to achieve in their particular domain, be it a hotel, a lending library, a bank or a manufacturing process.

Example 2

Suppose, after a hard day's work, you decide to make yourself a hot drink. What do you do to achieve that goal? What are the actions and how do you organise them?

We start with a process called *make hot drink*. At the centre of this task is boiling water in a kettle. Usually, there are two steps.

- 1 Fill the kettle with water.
- 2 Boil the water.

There are also some decisions to be made, such as the choice of beverage. We will consider instant coffee (as a particular scenario) to begin with. Finally, you need a cup to drink from. Figure 1 shows an activity diagram that models the process of making a hot drink of instant coffee with milk. It shows one way of preparing a cup of coffee. '... Pour boiled water into a cup containing coffee. Add milk. Drink coffee.' Of course, there are other ways of achieving the same result, and this will be true of problems in other domains.

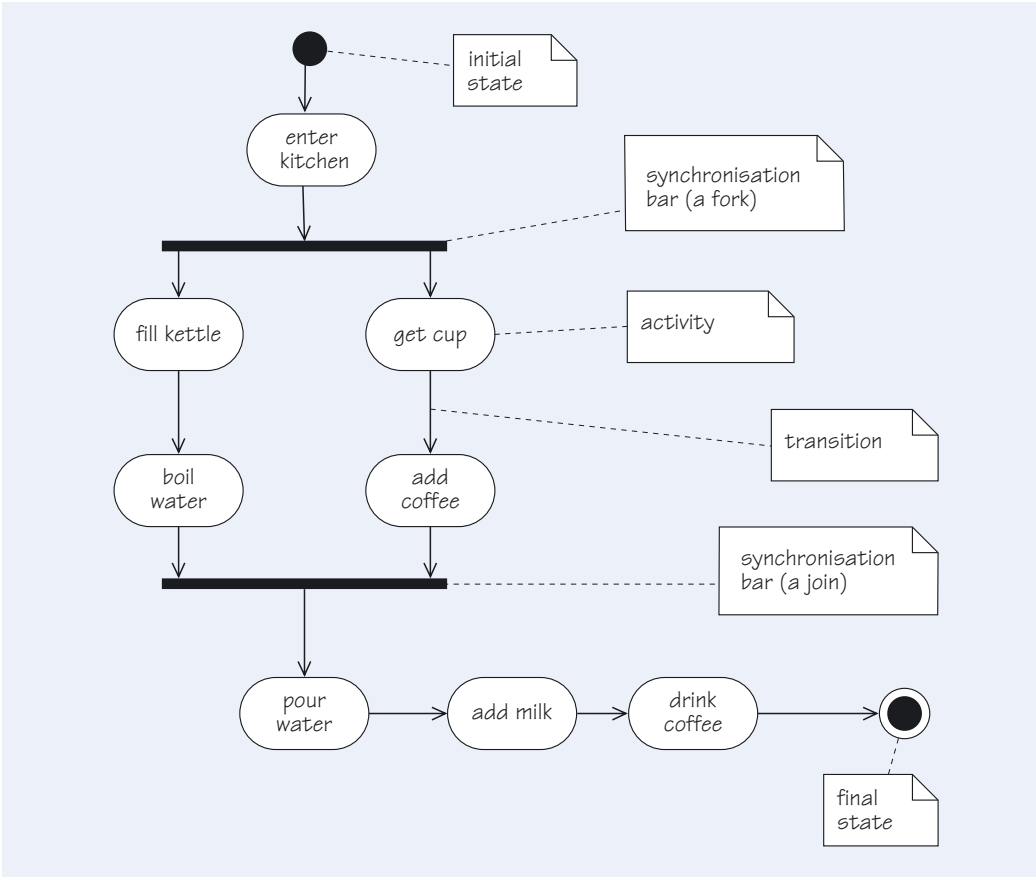


Figure 1 Making a cup of coffee

The basic elements of an activity diagram are activities and **transitions**; the completion of an activity triggers an outgoing transition. Similarly, once the incoming transition has been triggered, an activity may commence.

In an activity diagram, activities are shown as rounded boxes, while transitions are shown as lines with arrows. There are two predefined activities, *start* and *stop*, which are represented as filled circles, as shown in Figure 1. A diagram may include more than one stop node if it makes it clearer.

The order of the activities of boiling the water and adding coffee to a cup is immaterial, and so we show them as two concurrent flows in the diagram. The model shows that we expect some boiled water in the kettle and some coffee in a cup before we pour the water into the cup. Figure 1 shows two **synchronisation bars**, denoted by thick horizontal lines. The upper one denotes a **fork** and the lower one denotes a **join**. The upper bar allows for separate activities to be carried out in parallel. Between the two synchronisation bars the two separate strands can be performed concurrently. You could, for instance, get a cup and then add some instant coffee to it before filling the kettle with water and turning it on. Alternatively, you might get the water boiling before getting the cup and coffee. Or you could invite a friend to add the coffee to the cup while you fill the kettle! The lower bar prevents you from carrying on until both strands that lead into it are completed. ‘You cannot add the water until it has boiled and there is coffee in the cup.’

Strictly, the term used in UML 2 for a unit of behaviour is *action*, while an *activity* comprises a series of actions and is represented by a whole activity diagram. In this course, however, we will continue to use the term *activity* for each unit of behaviour.

Example 2 showed one way of making coffee. You can model alternative ways of making coffee in different activity diagrams. The ability to model alternatives allows you to investigate different ways of working. For example, you might model alternative ways for library staff to process the lending and returning of copies of books in order to provide a gain in efficiency.

In an activity diagram, the transition from one activity to the next is denoted by a stick arrow; it signifies that one activity has been completed and that it is time to move on to the next one. For example, in Figure 1, once you have filled the kettle, you can boil the water by turning it on. We have assumed that most people prefer hot rather than cold water when making coffee. So you wait until the water has boiled before you add it to the cup containing the coffee.

When there are alternative ways out of an activity, it is essential to indicate under which circumstances each transition is taken. Since the flow in an activity diagram is intended to proceed from one activity to the next, it would not be sensible to have two or more possible outflows without giving some idea of how the individual who is carrying out the activities might choose between them. UML provides **decision nodes** to represent alternative ways out of an activity; a decision node is drawn as a diamond. Figure 2 shows how decision nodes are used to deal with the possibility that the coffee cup is dirty.

In Figure 2, the flow between activities is constrained with a Boolean test, known as a **guard**, which is represented inside a pair of square brackets. Each of the transitions leaving the first decision diamond has a guard to determine which path should be taken under a given condition. One outgoing transition should always be taken; this means that exactly one of the relevant guards must always evaluate to true. In the diagram, if the cup is clean, you take one path, but if the cup is dirty you take the other path. In an activity diagram, the only label that can be applied to a transition is a guard that is intended to constrain the flow from one activity to the next.

Guards are associated with transitions. A transition takes place only if its guard evaluates to true at the time the flow of control reaches that transition.

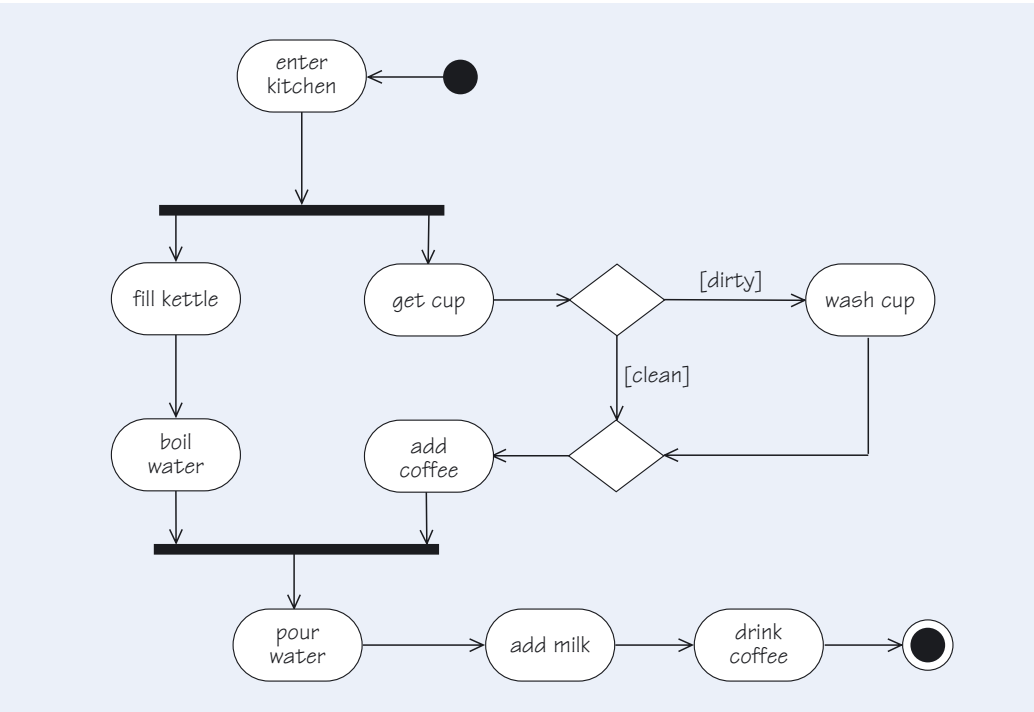


Figure 2 Using decision diamonds

A diamond shape is also used as a **merge node**, which brings together alternative flows, as shown in Figure 2. A merge node has a single outgoing flow.

There is always only one transition out of, and only one transition into, an activity. This is enforced by decision and merge nodes for alternative outgoing and incoming transitions, and by synchronisation bars for concurrent outgoing (fork) and incoming (join) transitions.

As you find out more about the domain in question, you may need to perform a further decomposition of the activities in your original diagrams. For example, the activity *fill kettle* in Figure 1 may be better represented by showing the steps involved in filling a kettle, especially when you have more than one type of kettle to choose from.

Modelling alternatives helps you investigate possible improvements in a given process. However, the investigation of alternatives for process improvement purposes requires knowledge of the problem domain. Making coffee is probably a very familiar domain to you. In practice, the domains you are likely to meet will not be as familiar.

SAQ 2

- (a) What causes a transition in an activity diagram?
- (b) What is a synchronisation bar, and when is one used in an activity diagram?
- (c) Figure 1 represents a particular way of making a cup of coffee. Suggest a reason why the activity *add coffee* has been placed before the joining synchronisation bar rather than immediately after the bar.

ANSWER.....

- (a) A transition in an activity diagram is caused by the completion of an activity.
- (b) A synchronisation bar is used to mark the point when two or more activities can take place independently (a 'fork'), or when a number of tasks must finish before continuing (a 'join').
- (c) When the kettle is full and you are waiting for the water to boil, there is some 'free' time that you can use to add coffee to the cup. Placing the activity *add coffee* after the joining synchronisation bar rather than before it would mean that you would have to wait to carry out the activity until the water had boiled, and the overall time taken for the task would be longer than that shown in Figure 1.

Activity diagrams and workflows

A business process is intended for a particular group of roles taken by people carrying out, and interacting with, the process.

An activity diagram can help identify the stages at which each role requires some interaction with the process. This is of particular benefit when we want to investigate the steps that people take in order to do their jobs. This flow of activities is known as a **workflow**. When modelling a workflow that involves more than one role, it is possible to identify which role is responsible for a particular activity. To do this in UML, we partition an activity diagram into **swimlanes** – one swimlane for each role, as we will now illustrate.

A role is the behaviour of a person participating in a process. One person may take different roles; for example, in a library the same person may take the role of a librarian and that of a member. That is why we talk about roles rather than people.

Example 3

Consider the library described in Example 1.

Book issue

Figure 3 illustrates the process of issuing a copy of a book in a library. The activity diagram has two swimlanes separated by a vertical line. The left-hand swimlane contains the activities that a library member performs when borrowing a book, while the right-hand swimlane shows the librarian's workflow for the same process (*issue copy of book*).

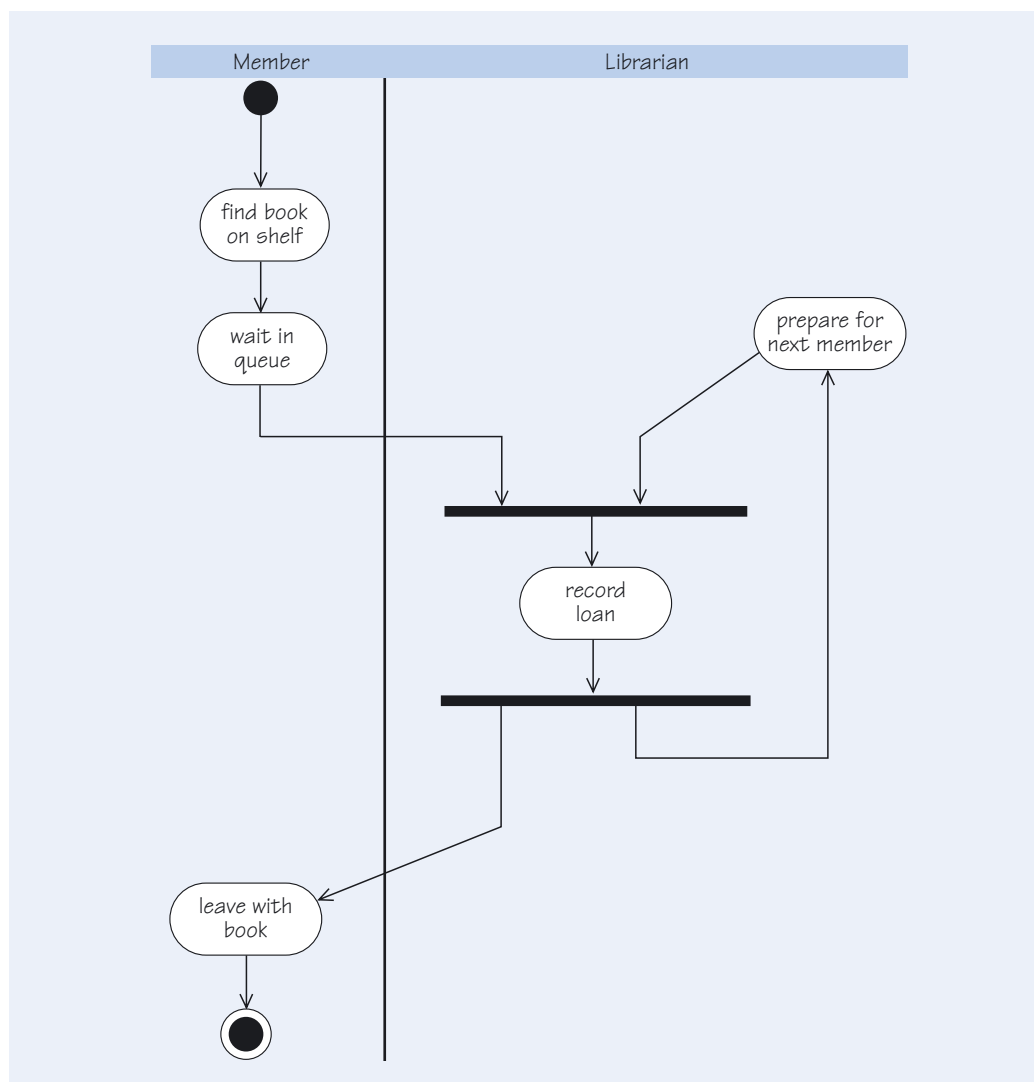


Figure 3 An activity diagram for issuing a copy of a book

The diagram shows that a member looks for a book on the library shelves and selects a copy to borrow. The librarian keeps a record of each copy that has been borrowed. So, the member must take the selected copy to a librarian for the loan to be recorded. There may be a queue of members waiting to take out books, and the librarian must deal with each one in turn.

Figure 3 shows that transitions may cross swimlanes in an activity diagram. We have assumed that after the librarian has recorded the loan of a particular copy of a book, they must perform some activity to prepare for the next member in the queue. The diagram shows that a loan cannot be recorded until the librarian is ready to deal with whoever is at the front of the queue. The

diagram shows the librarian's role as performing a repetitive business activity; for the purpose of illustrating the borrowing process, we are not concerned with other activities of the librarian, but only with those related to the interaction with each borrowing member.

So the activity *record loan* is likely to involve:

- ▶ identifying the particular copy of a book;
- ▶ identifying the member who wants the book;
- ▶ recording the loan, that is, creating a link between the member and the copy;
- ▶ identifying the period of the loan.

Book return

We will now consider how the books are returned. This is illustrated in Figure 4. Here, the member brings the book back into the library rather than selecting one from a shelf. As soon as the librarian has recorded the return of the book, the member leaves the queue without it. Figure 4 does not include a librarian activity for returning copies of books to the library shelves. To do this, we would need to know more about how restocking shelves takes place. Nevertheless, we have included an activity to put the returned book aside.

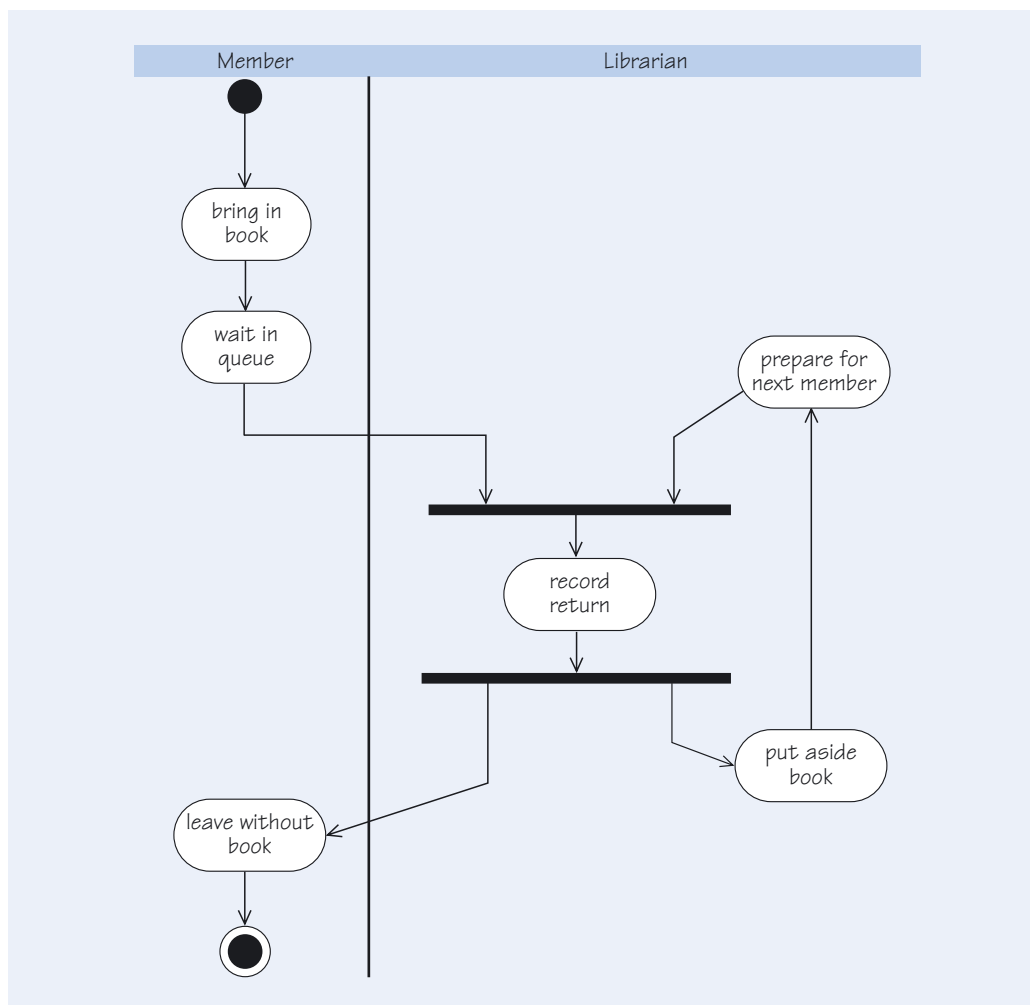


Figure 4 An activity diagram for returning a copy of a book

Although the question of overdue books was identified in the description of the library, we have not addressed handling fines. If we were designing a library system in real life we would need to investigate the fines system at this point, by asking the library staff. However we will not be pursuing the issue of the fines system here.

Figures 3 and 4 could imply two separate queues. Alternatively there might be only one queue for both borrowers and returners, in which case the librarian has to find out from the person at the front of the queue whether they wish to borrow or return a book. We pursue this further in Exercise 3.

SAQ 3

- (a) How does the partitioning of activities into swimlanes help us understand a set of activities?
- (b) Indicate one reason for modelling a workflow in an activity diagram.

ANSWER.....

- (a) Swimlanes group activities associated with different roles. Each swimlane shows who is responsible for which set of related activities.
- (b) Activity diagrams help to identify which role is responsible for which activities in a business process. An activity diagram can help identify the stages at which each role requires some interaction with the process. When you are modelling a workflow that involves more than one role, it is possible to identify which role is responsible for a particular activity.



Exercise 1

Figure 1 shows one way to model the preparation of a cup of coffee with milk. Redraw the figure to allow the choice between coffee with milk and coffee without milk.

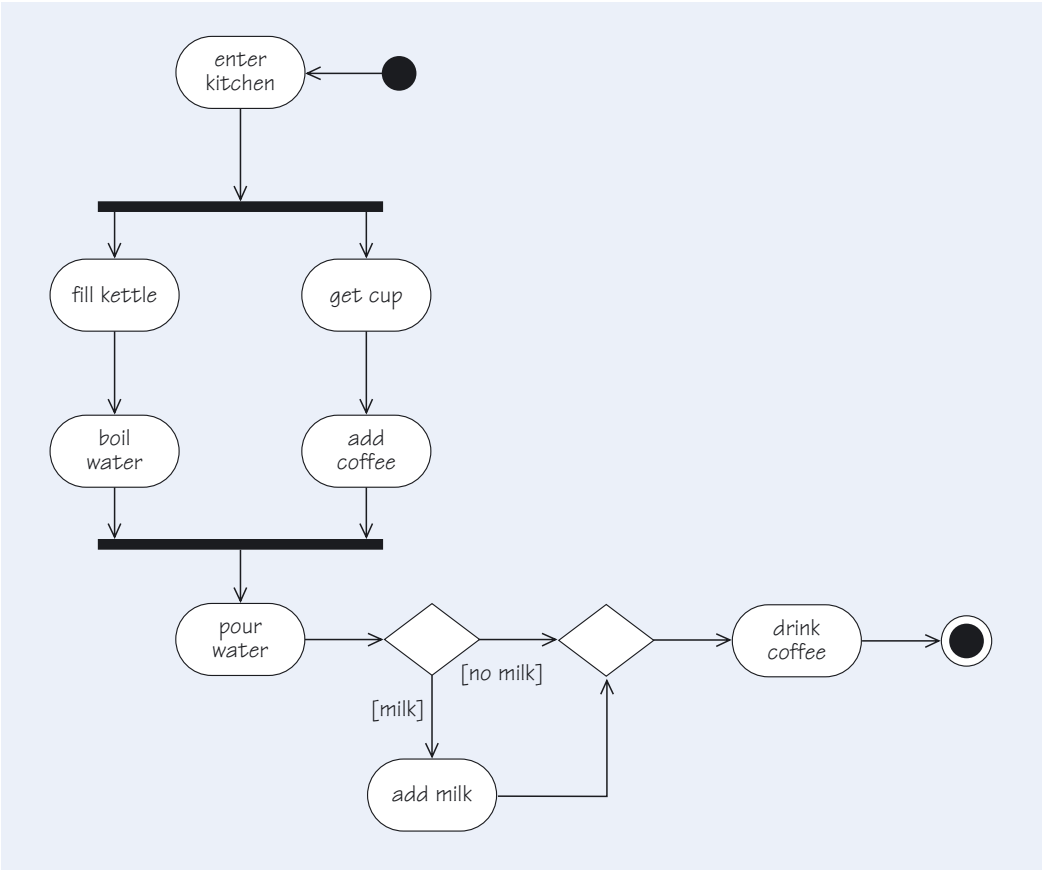


Figure 5 Extending the coffee-making activity

Solution

We can add a decision node to control the addition of milk to the coffee as shown in Figure 5.

We can choose either to add milk *or* not to add milk. This choice gives rise to alternative paths to the final activity of drinking the coffee. In contrast, the joining synchronisation point requires that the water has boiled *and* that there is coffee in a cup.

Note that with synchronisation bars, both paths are taken and execute concurrently; with decision and merger nodes, exactly one path is taken and there is no concurrency.

Exercise 2

Draw an activity diagram to illustrate the preparation of a pot of tea, which replaces the activities relating to coffee shown in Figure 1. The new diagram should allow the choice of both milk and sugar when the tea is poured.

Solution

There are a number of concurrent activities we can model that allow a fairly efficient workflow. The activity diagram in Figure 6 shows one of the ways of making a pot of tea. Note that we have not represented all possibilities, such as warming the teapot before use, or the use of a spoon when milk and sugar are involved. The activities of choosing whether or not to add milk and sugar to a cup, end with a diamond that joins together the paths representing the different choices. The convention is that the paths out of a decision node should be brought together at some stage at a merge node or activity. For example, the high-level activity of deciding between sugar and no sugar



has been expanded in such a way that there is only one path into the decision node and one way out from the merge node.

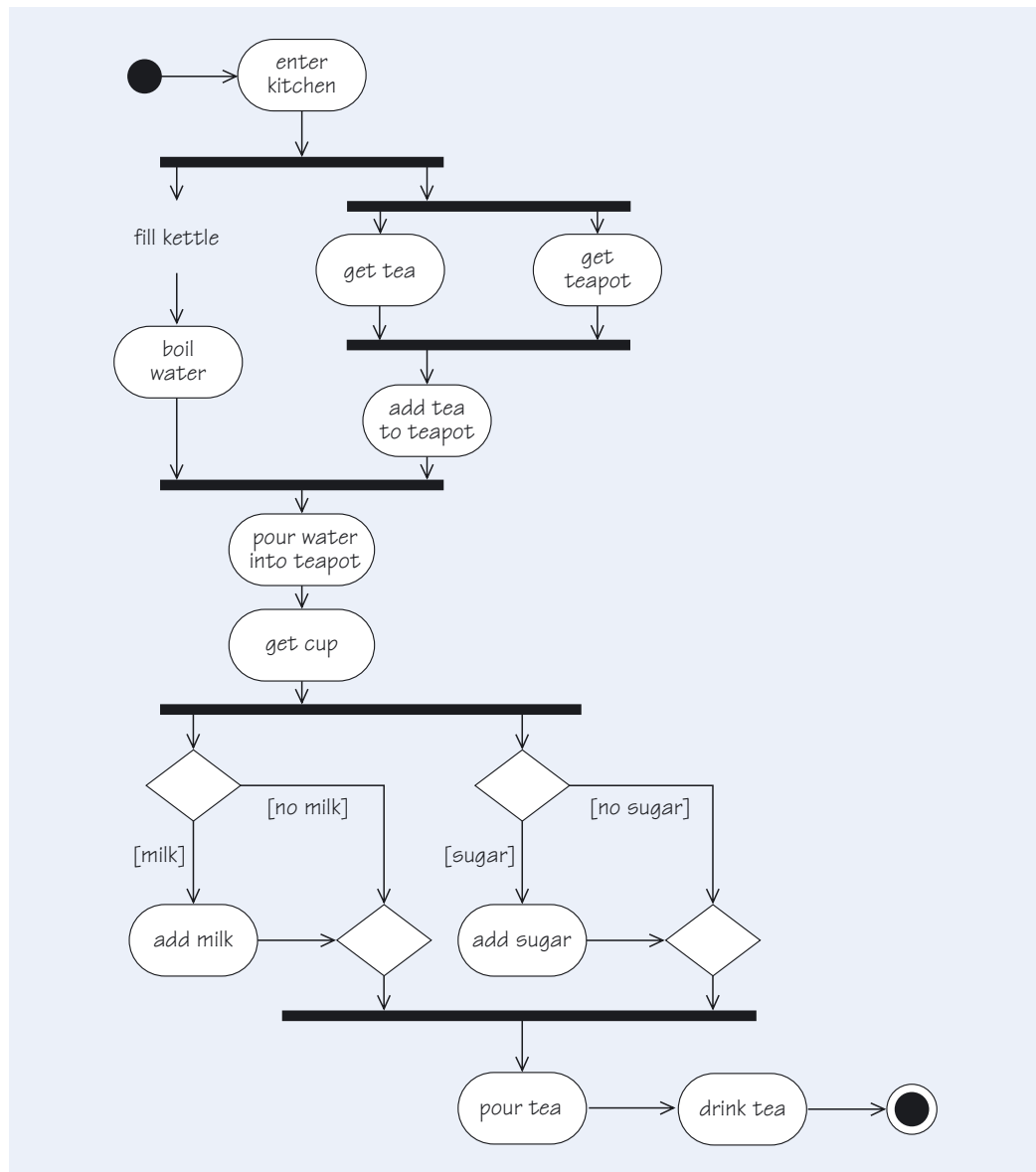


Figure 6 Making tea



Exercise 3

In Example 3, you saw one way to represent the borrowing and returning of books in a library. Use Figures 3 and 4 to produce a single activity diagram that models the borrowing and returning of books, based on the following assumptions.

- ▶ You do not need to consider fines for overdue books.
- ▶ The librarian places returned books onto a trolley (which will be used to carry the books back to the library's shelves at certain times during the day – an activity which need not be included at this stage).
- ▶ There is just one queue to deal with members.

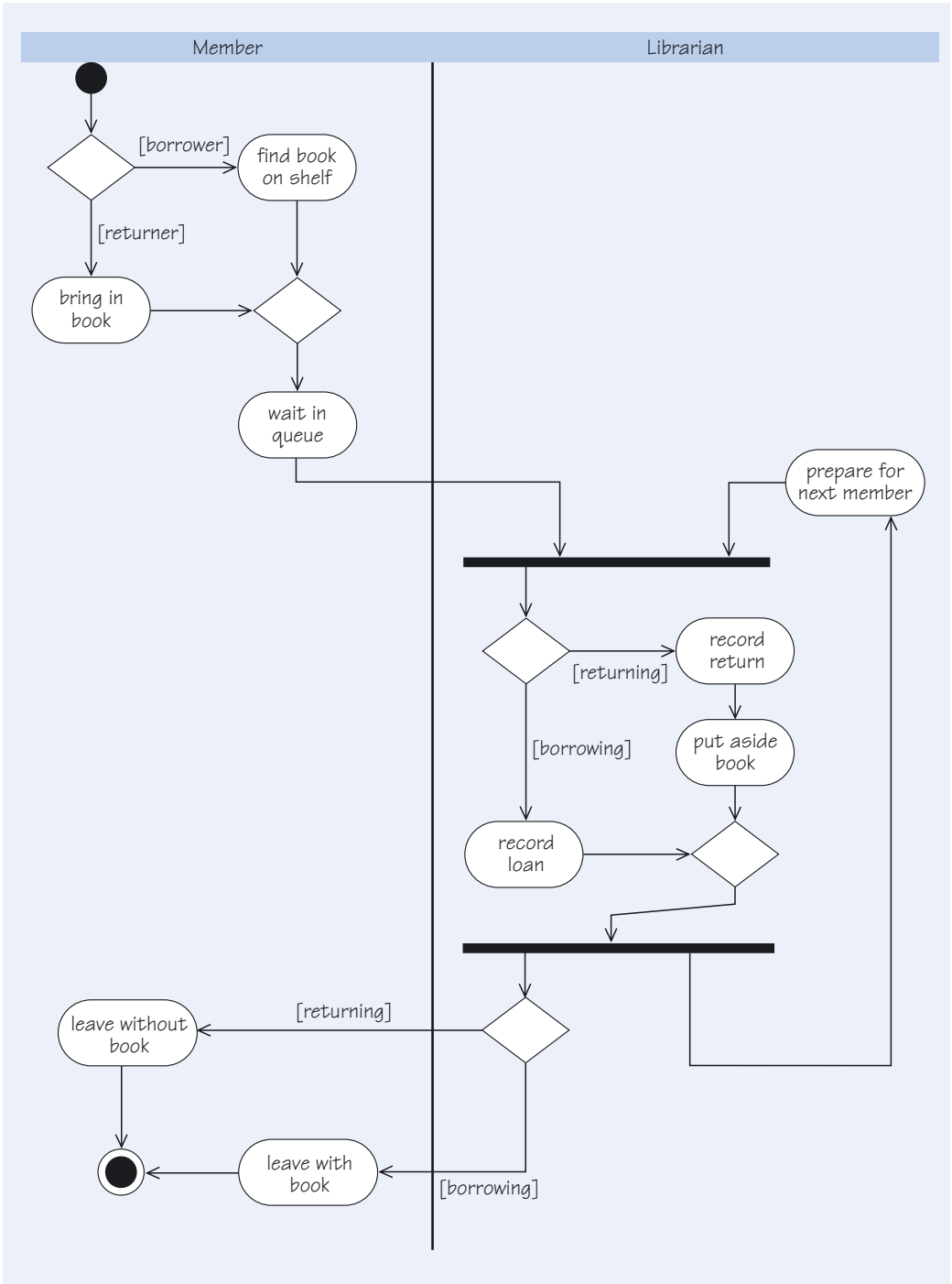


Figure 7 Borrowing and returning books to the library

Solution

In the combined activity diagram, we must use a series of decision boxes to control the flow between the different activities for borrowing and returning books. Figure 7 includes all of the activities from Figures 3 and 4.

We have made several assumptions here, but in reality we would ask the librarian what happens in practice.

A cautionary note should be heeded at this point. The activity diagram is becoming complex and more difficult to comprehend: keeping the two original processes separate may be the better course of action.

2.3 Summary of section

Business rules govern the functioning of a business, and business processes define the activities that take place, their ordering and the roles required to carry them out. Business processes can be represented by activity diagrams.

Activity diagrams are a way of understanding the workflows within the business domain. They allow you to record the things that users do or want to do. From a simple problem description, you can show the flow of control between the activities that are carried out by a role.

3

Simple use case models

This section introduces a means of defining what a proposed system should do from a user's perspective; this can provide a basis for a contract between the customer and the developer.

On completion of this section you should be able to:

- ▶ describe the role of use case modelling in software development;
- ▶ draw a use case model from a simple problem description;
- ▶ identify which actors are appropriate to a use case;
- ▶ identify pre- and postconditions that specify the meaning of a use case;
- ▶ identify scenarios for a use case.

3.1 Use cases

Your aim is to construct a software system that will meet the needs of its users. Hence you must identify 'who does what'. To do this, you search or explore the problem description, any domain models and the initial set of user requirements to determine the people involved, the work that they do and the events that trigger some work to be done. During that exploration, you are likely to identify a variety of people, some of whom may be playing a number of different roles and may be associated with different business events. Your aim should be to find out who will really do the work – or who is associated with the events that trigger the work – that will be helped by the proposed software system. You should ask yourself who the actual users will be and what tasks they must perform with the aid of your software system.

UML employs the term **actor** to identify a user of a system. An actor can also be an external software system or subsystem. A shop, for example, might have a point of sale system for the goods it sells to customers. If customers pay by credit card, that point of sale system might communicate with a credit-checking system owned by the customer's bank. So the credit-checking system is an actor with respect to the point of sale system. In other words, an actor is someone or something that is external to your intended software system. In a hotel, for example, one aim is to allow potential guests to make reservations. In this case, a guest is a potential actor – which we will name *Guest* – with respect to the intended software system. At this stage, it is not clear whether this is an actual guest or someone acting on behalf of that guest; this is something to be resolved in due course during the development project.

Use cases are a way of capturing functional requirements. A **use case** is something that an actor needs to do with the help of a software system; it should lead to a result that brings some observable value to that actor. If making a reservation is something that a guest can do with the new hotel system, the phrase *make reservation* would be a reasonable name for the use case for the actor *Guest*. The phrase that describes a use case can, of course, hide quite complex behaviour.

There are a number of possible things that can happen when a guest makes a reservation. For example, a guest may or may not want to take up the hotel's offer. In practice, the name of a use case reflects the task's goal. A typical *Guest*, for example, expects that *make reservation* is going to succeed.

A **use case diagram**, in UML, is the way to record and communicate your understanding of the use case view of a system. Figure 8 is a simplified use case diagram for the hotel reservation subsystem. The main elements in the diagram are:

- ▶ the actors, represented by stick figures;
- ▶ the use cases, represented by ovals;
- ▶ the relationships between actors and use cases – their **associations** – represented by lines.

An association is a relationship between two types, in this case between actor and use case, which represents all the links between instances of those types.

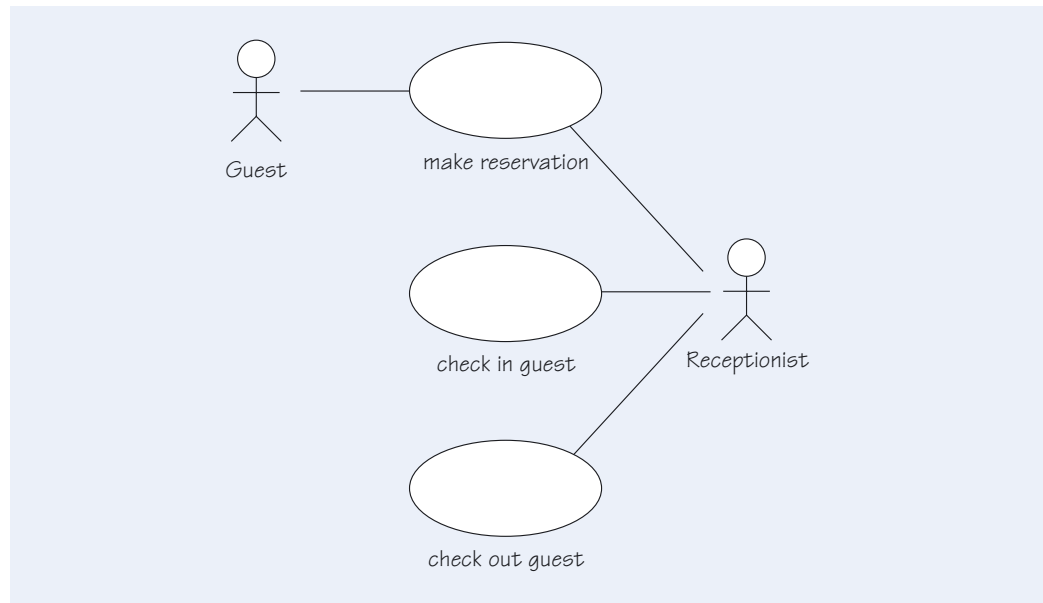


Figure 8 A use case diagram for a hotel chain

3.2 Emphasis on the user

The main purpose of use cases is to help with the elicitation and modelling of requirements. In *Unit 2*, we described how the elicitation of detailed software requirements can follow from a description of a use case. Use cases define the scope of a system, as they represent the interaction of a system with its environment. They also help with project planning and quality assurance, as you will see later. Furthermore, they act as a discussion tool between developer and user, and offer a common language for agreeing on the functions of the software system.

One use case represents one of the possible ways a system can be used. The use cases for a software system are a record of the intended user-visible behaviour of the system. This behaviour is what the software system does when responding to the events that arise from its interactions with a set of actors. The people who use the software system will be one group of actors, but there may be other software systems and devices that must interact with the intended software system. Do not be misled by the stick-person notation for an actor, as used in Figure 8. Representing other (sub) systems as actors lets you focus upon your area of concern, leaving the internal structure and behaviour of the other (sub)system to someone else.

The contents of the use case oval represent some **task** – or *coherent unit of functionality*, as UML defines it – which the software system is meant to perform. The detailed description of each use case can be held inside a CASE tool or a document. A use case diagram should be a concise representation of the main tasks and their relevant users.

The simple notations for the elements of a use case diagram, like those in Figure 8, are intended to be intuitive, even for a lay person who is unfamiliar with the notation and who is unlikely to know anything about UML. It is possible to exploit this simplicity to represent the main functions of a particular business. In *Unit 5*, you will see how UML is used to represent the *business objects* that the business considers to be significant. For example, if your business were a lending library, the main functions to be represented in a use case diagram would be the borrowing of books, DVDs and CDs by its members. Copies of books, films and music are the things handled or used by people; they are examples of business objects.

Since it will take a finite time to record the borrowing or lending of a particular copy of a book, DVD or CD, the lending library may have a requirement on the number of such transactions that a new software system must be able to deal with in a given period (say, one transaction per second). Similarly, the system may have to respond to queries against the library catalogue within a given time. These are examples of *non-functional requirements*, which you encountered in *Unit 2*. No matter what non-functional requirements are asked for, the developer's goal is to be confident that all the critical requirements are understood before investing any substantial effort in the development. Therefore, since use case diagrams deal with functional requirements alone, you should ensure that a non-functional requirement is not 'lost', by associating each non-functional requirement with the use case it relates to. We adopt a template to document each individual requirement and related information; examples of this template will be shown in *Unit 4*.

Use case analysis is related to an activity called **user-centred design**, which is part of the discipline of human–computer interaction (HCI). User-centred design differs from other approaches to design because it focuses on people, their work, their environment, and how technology can be best designed and deployed to support them. Use cases employ a similar approach because use case analysis focuses on the user and approaches the system from the user's point of view. User-centred design involves a rapid, iterative process with comprehensive user evaluations. But, as with all approaches to developing software, there is no one approach that is 'best' for all problems.

User-centred design and use cases are more concerned with external functionality than with how internal components of a software system interact. In short, users are interested in what the proposed software system will do, not in how the developers will produce the software, provided that it meets their understanding of a 'good software system'.

SAQ 4

- (a) Name three aspects of software development where use case modelling can help.
- (b) Suggest a reason why use case diagrams are an aid to communication between user and developer.

ANSWER.....

- (a) Any three of the following: capturing and eliciting requirements; representing requirements; planning iterations of development; validating software systems.
 - (b) Use cases offer users an opportunity to understand the system without the need to understand all of UML since the use case notation is relatively simple. This provides a mechanism that enables both developer and client to share a common understanding of the system, as long as the developer provides some text to demonstrate his or her understanding of the problem.
-

3.3 Stepping from the problem to its solution

As you saw in Section 2 of this unit, the problem that a developer needs to understand is in the world outside the computing system. In terms of the models described in Section 2, the analyst documents his or her understanding of the problem domain with activity diagrams. The next step is to understand the requirements for a system that will be a solution in that domain. The requirements are the basis for development and the specification of that solution. As we saw in *Unit 2*, the relevant business events from the domain can be associated with a use case for the system to be developed. The context of the new system will be defined by the use cases for all relevant business events. In a problem that involves an existing software system, you might find a set of use cases that represents the existing software system in the problem description.

The system boundary is an important conceptual line that separates what we are interested in from the rest of the world. In drawing the boundary around the software system represented in your use case diagram, you are setting the scope of your solution. In UML, a solid box drawn around the use cases with the actors located outside it represents the system boundary. Figure 9 shows a boundary for the hotel-chain use cases.

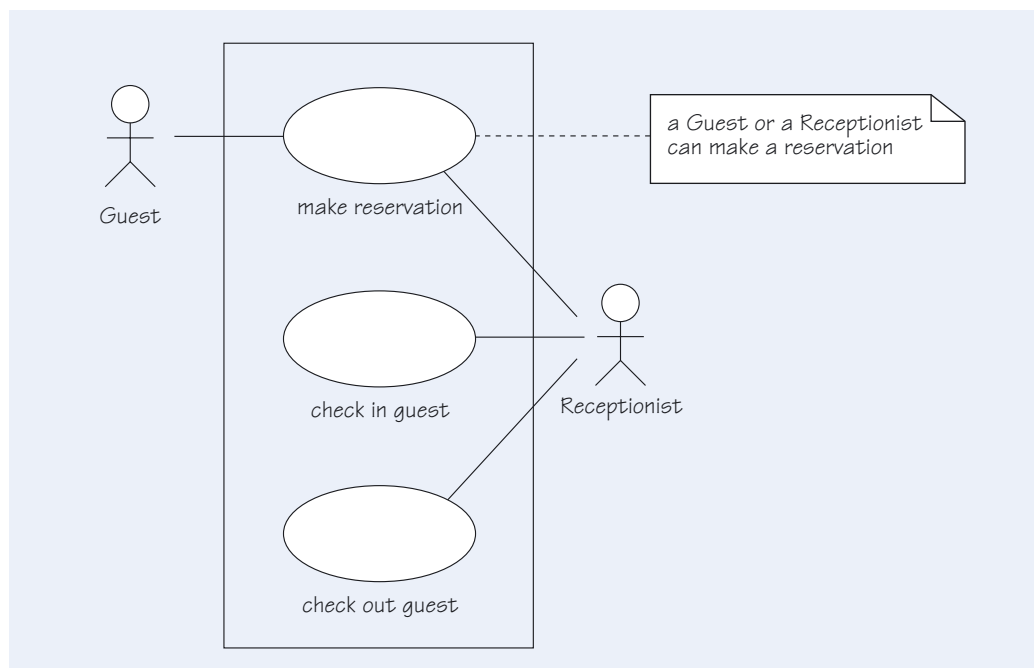


Figure 9 A use case diagram for a hotel chain, showing the system boundary

The previous figure, Figure 8, illustrates a problem. From the diagram alone, it is not clear which of the two actors, *Guest* or *Receptionist*, initiates the *make reservation* use case. It may even be that both are needed to complete the use case successfully. In fact, potential guests will be allowed to use the internet to make a reservation, but in other cases the receptionist performs the task. UML provides several ways to deal with the problem. The simplest, which is shown in Figure 9, is to use a textual note to record this observation, and then refine the model when you know more about the use case. In UML, a note is shown as a rectangle with the top right-hand corner folded down. A dashed line is used to attach the note to the model element to which it refers.

In simple use case diagrams, it is common to leave out the box showing the system boundary, since it is not a compulsory model element. Unless we are discussing a complex problem that might involve several subsystems, we will not show the system boundary.

SAQ 5

What is the purpose of a system boundary? Is it always necessary to draw one in a use case diagram?

ANSWER.....

The purpose of a system boundary is to identify a single system, distinguishing between the internal and external components. Typically, the external components are the actors, and the internal components are the use cases. UML says that the system boundary is optional.

3.4 Actors

Iteration is a natural part of the modelling process. It does not matter whether you start by looking for the actors or the use cases. We have chosen to begin with the actors, since it is a way of expressing the system boundary implicitly and identifying the different views that need to be taken into account. In practice, you are likely to find that the actors are to be found in the roles that people play as employees in the problem domain, such as the hotel’s receptionist or manager. You have already encountered these roles in the activity diagrams for the business processes of the domain.

Actors are not intended to represent particular individuals; rather, they tell us about a particular role that someone or something might adopt in order to interact with a software system. For example, someone who works as a receptionist in one hotel might want to stay in another hotel as part of his or her holiday. That is to say, he or she will adopt the role of a guest.

If you have not drawn activity diagrams for the business processes, you need to know where to start to find the actors. Given the relationship to organisational roles, you might identify the actors by looking for the groups of people who are going to use the software system in order to do their work. It might be easy to find those who perform the main tasks, such as the receptionists who work on the front desk of a hotel. But it might be harder to find those who need the software system in support of their administrative or maintenance tasks. For example, would the maintenance engineers and cleaners in the hotel interact with your intended software system directly?

In the case of the hotel chain, it is likely that you will need to pass information about a guest’s stay to an accounting system. At some later point, you may be asked to provide an interface to the restaurant side of the hotel in order to associate the costs of any meals with the guests who ate them. When a guest leaves the hotel, there may be a requirement to collect payment for the guest’s stay from an external banking or credit card system. In each case, you should consider whether or not there is some value in an exchange between the use case in the software system, which you are to develop, and any identified external system. For each one that you have chosen to include, treat it as though it were an autonomous black box. You do not need to know how it works. You only need to know about the interface of the external system.

It is important to distinguish between an actor and the way that actor communicates with the software system. For example, when analysing a system, you should not be concerned with the mechanism used by the receptionist to check guests in and out of the hotel system. It could involve the use of a keyboard and a mouse to interact with a series of screens on a PC. It might even include a TCP/IP connection or voice recognition software. You should concentrate on the meaning of the stimuli and responses for any given use case, not the communication mechanism that is used. That mechanism is part of the solution, which you intend to provide.

In situations where two or more actors are associated with a use case, one of them must initiate the actions. The other actors will play a passive role. For example, when a guest checks into a hotel in person, the receptionist performs the check in process, and the guest is passive. When it comes to describing a use case, treat the software system as a black box, as in the case of an external system. It will accept stimuli from the actors and generate responses.

SAQ 6

- (a) Explain why the actors in a use case diagram do not represent actual individuals.
- (b) Suggest a guideline that will help you decide whether or not to include an interaction with an external system on your use case diagram.

ANSWER.....

- (a) An actor in a use case diagram represents a particular role that an individual might play when interacting with a software system. For example, a receptionist checks guests into and out of a hotel (see Figure 9). But it could be that the person who works as a receptionist at one hotel becomes a guest at another hotel in the chain and hence takes on another role.
- (b) Show interaction with an external system if the use case needs to communicate with the actors that represents the external system.

3.5 Describing use cases

Before you can describe detailed requirements, or carry out a sensible estimation and planning process for the whole project, you need, among other things, a good idea of what each use case means. To get a feel for what this might entail, look again at Figure 9, which shows a simple use case model for a hotel chain. The main activities involve making reservations and checking guests in and out of rooms.

What do the use cases *make reservation*, *check in guest* and *check out guest* mean? No doubt, using your own experience of reserving rooms at hotels, the names of the use cases are quite indicative of what they mean. However, you should never rely on intuition or personal experience, but rather create a description of the use cases that you and the customer can agree upon. For example, a description of the *check in guest* use case might be as follows (for a particularly simple system).

Upon arrival, each guest provides the reservation number for his or her reservation to the hotel's receptionist, who enters it into the software system. The software system reveals the details of that reservation so that each guest can confirm them. The software system allocates an appropriate room to that guest and opens a bill for the duration of the stay. The receptionist issues a key for the room.

Having read this description you are probably thinking of all the deficiencies it contains. In real life, you would of course check with the customer to clarify that the description contains all the necessary information. However, for the sake of our imaginary example, we will assume the description given is sufficient.

The information relevant to a use case needs to be recorded. We look now at what the record should contain, and draw your attention to the following observations about the requirements of the *check in guest* use case.

- There is a condition, known as a **precondition**, that must hold before a room can be allocated to a guest. This is as follows.

There must be a reservation for the guest, and there must be at least one room available (of the desired type), and the guest must be able to pay for the room.

- There is a condition, known as a **postcondition**, that must hold after a room has been allocated to a guest. This is as follows.

The guest will have been allocated to a room for the period identified in the reservation, the room will have been identified as being in use for a specific period, a bill will have been opened for the duration of the stay, and a key will have been issued.

In other words, we have captured the meaning of *check in guest* in terms of two statements: one that must be true before the use case can be carried out (the precondition); and one that must be true once the use case has been completed (the postcondition). The advantage of describing a use case in terms of a precondition and a postcondition is that, when you go on to elaborate the use case in more detail, these same conditions must still be satisfied; they will continue to specify the starting point of the use case and its required outcome.

Notice that such a specification does not say *how* a use case must be performed, but simply what conditions must be satisfied.

The two descriptions – the first, which is entirely prose, and the second, which is more formal using pre- and postconditions – are both descriptions (specifications) of the *check in guest* use case.

Scenarios

The purpose of a use case is to meet the goal of its associated actor or actors, such as a guest making a reservation with a hotel. So a use case should include all the steps necessary to meet that goal with the aid of the software system. For example, if it is necessary to check the availability of rooms in the hotel for the desired length of stay before accepting a reservation, then we expect the use case to perform that check. In general, a use case contains a narrative about the flow of events that specifies a particular use of the software system.

A **scenario** illustrates one particular way the use case can unfold, by the sequence of interactions the actors have with the system. You can think of a scenario as being an *instance* of a use case, much as an object is an instance of a class.

For each use case, we normally start by considering a main scenario in which the goal of the use case is successfully achieved – the ‘sunny day scenario’. We then go on to consider other scenarios in which things go less smoothly: the goal is not achieved, or there has to be some variation in the sequence of actions that is carried out.

Here, for example, are two scenarios for making a reservation at a hotel – first the main success scenario, then an unsuccessful scenario in which no reservation is made.

Main scenario

The guest wants to reserve a double room at the Ritz Hotel for 14 July. A double room is available for that date, and so the system makes a reservation for the guest.

Unsuccessful scenario

The guest wants to reserve a single room at the Savoy Hotel for the first week of August. There is no single room that is free for seven days in August, but there is one room available for four days and another of the same type for the following three days. The system presents that option to the guest, who rejects it.

Both scenarios are possible instances of the *make reservation* use case. Their interactions and outcomes are different. In the first, there is a description of the use case leading to a successful outcome. In the second, there is an exception to the main

success scenario. Exceptions to the normal behaviour for a use case are common, especially where actors decide to cancel a use case without completing it. However, the common theme among all the scenarios is the intent of an actor to reach the goal defined by a use case. In the unsuccessful scenario above, the guest was trying to make a reservation at the Savoy Hotel, but rejected the option of changing rooms during the stay. This sort of situation is common, and a use case must always cater for any unusual or alternative courses of action that are likely, as you will see in the next section of this unit.

You could start with the simple identification of a use case and its main success scenario, and refine or adapt it during the process of development. You will need to decide the way in which you record the information for each use case – not just for the main success scenario, but for all the relevant scenarios. At its simplest, you can record a textual description (narrative) that details each scenario together with its outcome.

What a use case description should contain

Remember that your description of a use case expresses what the software system should do without constraining how it should do it. Since the description takes an external viewpoint, all the behaviour is in the form of observable results. While the structure and format might vary among the different development processes, we suggest that you include the following items as a minimum:

- ▶ a unique identifier for the use case, to allow traceability throughout development;
- ▶ the name of the actor that initiates the use case;
- ▶ a short description of the goal of the use case;
- ▶ a single sequence of steps that describe the main success scenario – you may find it helpful to number these steps for traceability, for identifying the detailed software requirements that they generate, and for cases where you need to identify any extensions or variations that occur as a result of the other scenarios of a use case;
- ▶ a textual description of the pre- and postconditions.

In some circumstances you may have to add other information. For example, the identity of the authors may be required where there is a large team of developers. In a risk-driven process, you might be required to record an assessment of the risks, assumptions and outstanding issues to support the decision-making process. It will generally be useful to record the things that the authors of a use case assumed to be true during their analysis.

Opinions vary about the correct format of the description of a use case. One development process might require a detailed structure with tightly controlled phrasing and numbering of each item in the description. Another might place few or no limitations, such that each use case reads like a story, with a beginning, a middle and an end. You should use the language of the domain to formulate the use cases, as they should be understood by both the developer (as supplier) and the users (as customers). Both parties need to have a clear understanding of what is captured in each use case and of what it means.

On an individual project or within a particular organisation all the developers would work to an agreed common format to document use cases. In the process followed in this course, we will use the steps of scenarios to transform user requirements into more detailed software requirements. Table 1 illustrates the style we follow in the course to structure and record a use case.

Table 1 A textual description of a use case in the hotel domain

Identifier and name	UC1 <i>make reservation</i>
Initiator	<i>Guest or Receptionist</i>
Goal	A room in a hotel is reserved for a guest.
Precondition	None (that is, there are no conditions to be satisfied prior to carrying out this use case).
Postcondition	A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions	The expected initiator is a guest, using a web browser to perform the use case, or a receptionist. The guest is not already known to the hotel's software system (see step 5).
Main success scenario	
1	The guest/receptionist chooses to make a reservation.
2	The guest/receptionist selects the desired hotel, dates and type of room.
3	The hotel system provides the availability and price for the request. (An offer is made.)
4	The guest/receptionist agrees to proceed with the offer.
5	The guest/receptionist provides identification and contact details for the hotel system's records.
6	The guest/receptionist provides payment details.
7	The hotel system creates a reservation and gives it a number.
8	The hotel system reveals the reservation number to the guest/receptionist.
9	The hotel system creates a confirmation of the reservation and sends it to the guest.

The main success scenario always contains a stepwise description of what happens when nothing goes wrong; for most use cases this will be the situation that occurs most often. It is assumed that the steps are performed in the order described with no concurrent behaviour. In the next section, you will see how each step can be used as a decision point to deal with exceptional circumstances. For example, what should be done if there is no available room for the desired stay?

SAQ 7

- (a) What is the relationship between a use case and a scenario? Give examples to illustrate your answer.
- (b) What is meant by a main success scenario?
- ANSWER.....
- (a) For each use case there is a set of possible scenarios. A scenario is an instance of a use case. A scenario describes a sequence of interactions between the system and some actors.

Here are two examples of scenarios. A member of a lending library wishes to borrow a book, and is allowed to do that as long as she has no outstanding loans. Another member wishes to borrow a book, but has exceeded his quota of the number of books he is allowed to borrow. In each scenario, the member wishes to borrow a book, but both the circumstances and outcomes of events are different in each instance. So a use case includes a complex set of requirements that the system must meet in order to cope with every eventuality.

- (b) The main success scenario shows the steps normally followed to achieve the stated goal of the use case. But there can be other scenarios for the same use case, each one having different outcomes depending upon circumstances.

3.6 Use cases in development activities

Representing requirements with use case diagrams

It is possible to represent requirements in a wide variety of forms: you might use a mixture of text, diagrams and mathematical expressions. In practice, you will find that each development process has something to say about the way developers should express a given set of requirements. We identified some basic guidelines for good documentation in *Unit 1*, and you should always consider the intended audience for any document that you produce.

You have seen how use cases can record the tasks that actors want to do using the proposed software system. In practice, the use cases are equivalent to the users' requirements that you plan to satisfy. For simple systems, there may be only one use case diagram, as in the hotel example of Figure 9. For large systems, you will need a number of use case diagrams, each representing just one aspect of the software system as a whole. Thus use case diagrams are an essential feature in the management of requirements. For simple use cases, we may equate the user requirement represented by the use case with one single software requirement. For more complex use cases, the corresponding user requirement will be refined into a set of more detailed software requirements corresponding to the steps of the scenarios.

Each use case diagram should focus on a clearly defined aspect of a software system, and should contain the actors and their use cases that are essential to express your understanding of that context. There is a balance to be struck concerning the level of detail within each diagram; too much or too little detail will detract from the diagram's usefulness in communicating your understanding of an aspect of a software system. You need to provide sufficient detail to be understandable, while still maintaining an adequate level of abstraction.

Use case diagrams show the behaviour that is required of the software system.

Eliciting detailed software requirements

Use cases can be used to elicit detailed software requirements as described in *Unit 2*. Each step in a use case scenario may correspond to one or more functional requirements. This will be illustrated in *Unit 4*; here we give a simple example.

Step 2 of the use case in Table 1 ('The guest/receptionist selects the desired hotel, dates and type of room.') corresponds to a set of detailed software requirements, as shown below. Each detailed requirement is associated with a scenario step, and, apart from its description, there is an indication of its associated fit criteria, as discussed in *Unit 2*. (SFR2 stands for Software Functional Requirement 2, UC1 for Use Case 1, and Step 2 for scenario step 2.)

SFR2: UC1, Step 2

Description: The system shall obtain a hotel name from the user.

Fit criterion: A valid hotel name shall be accepted.

SFR3: UC1, Step 2

Description: The system shall accept a range of dates from the user.

Fit criterion: A valid range of dates shall be accepted.

SFR4: UC1, Step 2

Description: The system shall accept a room type from the user.

Fit criterion: A valid room type shall be accepted.

Sometimes a simple use case corresponds to one requirement and needs no further detailing; however, for more complex use cases, decomposition into more detailed software requirements will be carried out as shown here.

Use cases as a planning aid

By the time you have completed your requirements elicitation and analysis, you should have both a good idea of what each use case means and an understanding of users' expectations of each one. If your chosen development process includes risk analysis, you should have recorded some assessment of the risks associated with each use case.

At this point, you will be in a position to produce a plan for the project as a whole. You have to decide the order in which to implement your use cases. Part of your planning task will be to assign a priority to each use case to indicate its importance with respect to the users. You can also use other factors to determine the priority of a use case; for example, some authors suggest risk, coverage and criticality as the main criteria (Krutchen, 2000). Risks include complexity of requirements, unknown required effort (involving significant research or new and risky technology, for example) and political issues. Coverage requires that the initial set of use cases should give an overall view of the system. This leads to an early definition of the architecture that will influence the rest of development.

Criticality is measured in terms of business value – the main business processes, for example. The main functions of the system should be covered early. The customer will be involved in the decisions concerning use case priorities. In many cases, there will be one or more immovable deadlines that will affect your choice. In some circumstances, you may be faced with the choice of removing functionality from your use case model in order to meet such deadlines.

Use cases and testing

Each detailed software requirement is tested against fit criteria. But it will also be necessary to carry out testing at a higher level of granularity, considering a series of use cases.

A simple and useful technique for validating a requirements specification is to 'act out' or 'walk through' each use case in turn. By involving the potential users, you can determine the extent to which your solution will meet their expectations. The *walk-through* technique can be used to specify acceptance tests, where you need to demonstrate that your software solution has met the specified requirements.

The same technique can be used for system testing, to verify that your design conforms to its specification. For each scenario in a use case, you can identify a set of specific instances to show that your solution can deal with each main success scenario and its variations and extensions. For example, you would design test cases to show that a guest can make a reservation for a room in the hotel. In the same way, you could also show that reservations are not possible when there are no rooms available for an identified stay in the desired hotel.

You may also choose to gather two or more use cases into a package for development as one increment of your project life cycle. See *Unit 7*.

Architecture is the overall structure of the software system – the main components, and how they are related to one another in order to guarantee certain qualities of the system.

Acceptance testing consists of a series of tests carried out by the customer in conjunction with the developer which act as a final check that the system does what the customer expects it to do.

SAQ 8

- (a) How do use cases help with requirements capture?
- (b) How do use cases help with the elicitation of detailed software requirements?
- (c) How do use cases help with development?
- (d) How is system validation performed using a use case?

ANSWER.....

- (a) Use cases help with requirements capture by assisting with the identification of actors and tasks in the system. For each actor, the set of use cases establishes what that actor requires from the software system. The association between an actor and a use case is about communication. Therefore you should also establish what the software system derives from the actors.
 - (b) Detailed software requirements can be associated with each step in a use case scenario.
 - (c) One of the difficulties that developers face is planning delivery times. Often a customer can place pressure on the developer to meet a particular deadline. However, it is the developer's job to indicate which criteria in the software system can be met under such constraints, and to elicit from the users those use cases that have the highest priority. The use case diagram helps by: providing an understanding of the complexity of each use case; determining which actors interact with each use case and to what degree; discovering which use cases carry the most risk; and estimating how long each use case is likely to take to implement. Understanding these aspects of the system can help developers plan the order in which the use cases should be developed, and provide an appropriate time frame. Several criteria – such as risk, coverage and criticality – can be used to help establish priorities of use cases.
 - (d) One way to validate a system is to use the walk-through technique. This is where each use case is examined in turn to assess whether the system actually supports the functionality required by the users – a process known as validation. The walk-through technique can also be used to elicit system tests where each use case is required to deal with a number of scenarios – a process known as verification. For each software requirement generated from a step of a scenario, the fit criterion helps to devise the test.
-



Exercise 4

A typical lending library keeps a stock of books for the use of its members. Each member can take out a number of books, up to a certain limit. After a given period, the library expects members to return the books that they have on loan.

When borrowing books, members are expected to hand their chosen books to the librarian, who records each new loan before issuing the books to the member. When a book is on loan to a member, it is associated with that member: possession of the book passes from the library to the member for a defined period. The normal loan period for each book is two weeks. If the member fails to bring the book back on or before the due date, the library imposes a fine.

In the proposed new software system, anyone should be able to browse the stock of books held in the library, but only a member will be able to reserve a book. The ability to browse and make reservations will be provided through a web-browser interface.

The system should also deal with the enrolment of new members for the library.

Draw a simple use case diagram for the proposed software system, and identify the constraints or assumptions that you make. (Ignore the issue of fines, because there is not enough information.)

Solution

A good first step is to establish the context for the proposed system by identifying the actors that surround it. When you have identified the actors, you can investigate the behaviour that each one will expect from the proposed system, which will eventually give rise to the use cases for your model.

Your task is to determine the roles that people will adopt in order to use the proposed software system. In general, the customer (the person who is buying the software) will have some say in who is a legitimate user. In this problem domain, the library owns the books. One of its employees, a librarian, is responsible for issuing books to a member. Typically, a librarian will also act on the member's behalf when recording a new loan. Hence we expect the librarian to be a key user of the new system.

Browsing through the library's catalogue is a common activity performed by both members and librarians. As a catalogue is only a kind of list that shows what the library offers for loan, it can be read by anyone: non-members are also to be allowed to browse the catalogue.

There appear to be two scenarios related to the activity of reservation. In the first, a member might want to borrow a specific book, but finds that all the available copies are out on loan. In the second, a member might want to make sure that at least one copy is available in order to avoid a wasted trip to the library.

We conclude that there are three basic actors:

- ▶ *Member*
- ▶ *Librarian*
- ▶ *Browser*

What is missing from the given description of the system is any indication of how people join the library and become members. We have been told that there is a possible fine for overdue books, but is there a fee to join the library? Do prospective members have to prove their identity? How do members prove their membership? So this is a potential use case that requires later elaboration

In addition, someone – a librarian, we presume – will be expected to make sure that the catalogue reflects the actual stock held at the library, and a corresponding use case is needed for this.

Our solution includes six basic use cases for the library software system:

- ▶ *issue copy of book*
- ▶ *return copy of book*
- ▶ *reserve book*
- ▶ *browse catalogue*
- ▶ *update catalogue*
- ▶ *enrol new member*

Figure 10 shows a possible use case diagram for the proposed system. You may have used different names on your diagram, and you may have identified more or fewer than six use cases. There is no one correct solution: there are many possibilities. You may have defined use cases that combine two or more of our use cases, but in doing this it is likely that you would have assumed knowledge of the internal structure of the proposed system. In a use case diagram, you should identify only the behaviour that will bring some discernible value to at least one actor.

In reaching your solution, you might have gone through several iterations. This is to be expected. Your first guess could have been done in your notebook, as opposed to using a CASE tool. In practice, you might alternate between pencil and paper and a

CASE tool. Whatever your chosen means of recording, focus on getting a useful result. A rough diagram that is useful is better than a pretty one that is not!

Finally, you should have included some notes with both the diagram and the elements in it. For example, in Figure 10, the *issue copy of book* use case has a note associated with it indicating the loan period. Notes are an excellent way to record constraints on the behavioural requirements.

Another use of a note is to indicate something that needs further attention or revision. Figure 10 shows that both members and librarians are associated with reserving books. A note records the fact that librarians can reserve books on behalf of members.

At some future point, once you have learnt more about the library, you would revise the use case diagram.

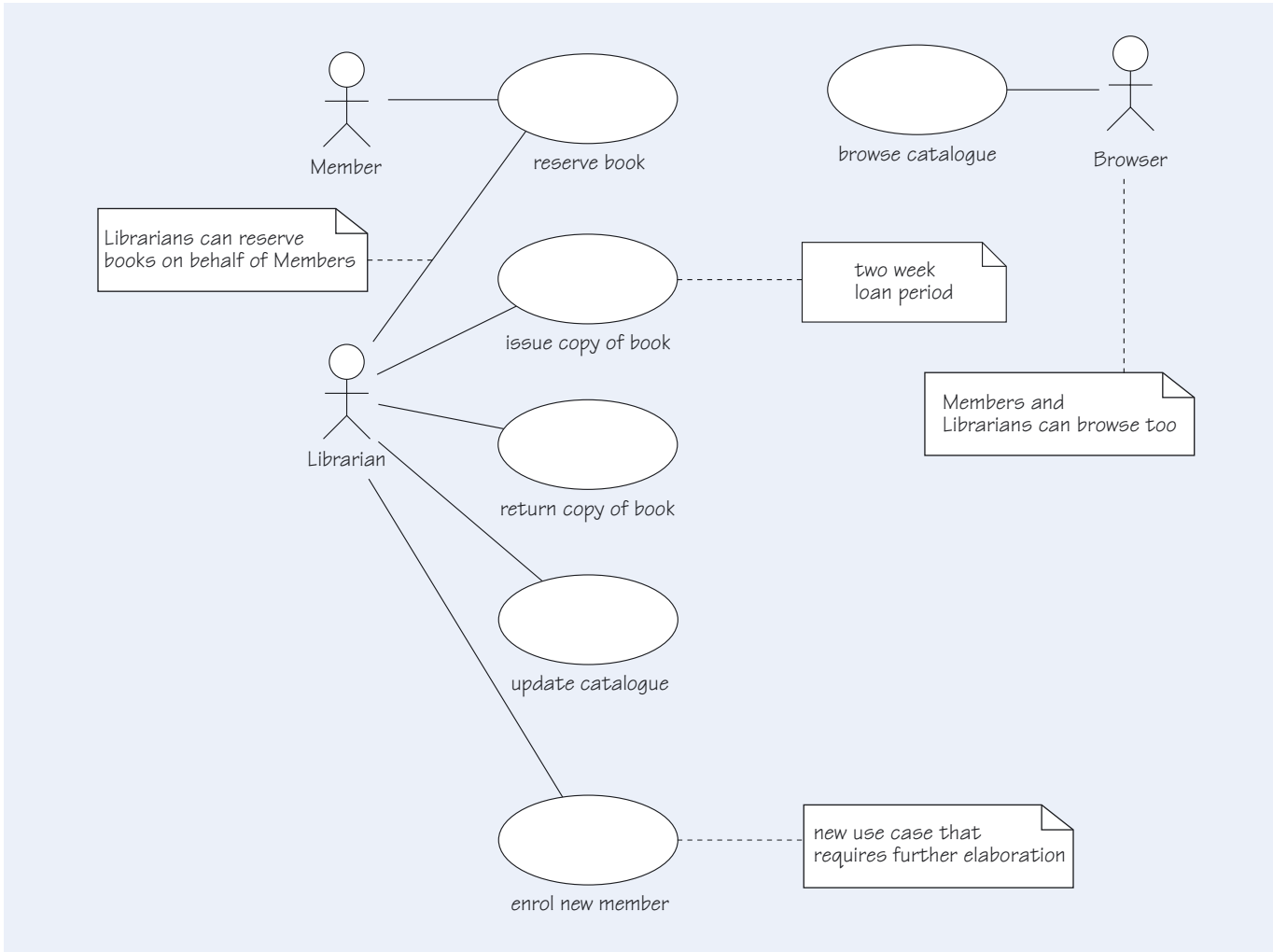


Figure 10 A use case diagram for a lending library



Exercise 5

Suggest a precondition and a postcondition for the *check out guest* use case given in Table 1. (You may also find the scenario of Table 1 useful.)

Solution

Precondition: the guest must be currently allocated to a room.

Postcondition: the room will have been designated as free to be reserved by another guest, and the guest will have paid his or her bill.

Exercise 6



Prepare a textual description of the *check in guest* use case that appears in Figure 9. Follow the example of Table 1, which shows the main success scenario and other criteria for the *make reservation* use case. As part of your working, identify any exceptions to the main success scenario. You will need to make some reasonable assumptions and should record them.

Solution

The actor for the *check in guest* use case is the *Receptionist*, as shown in Figure 9. You need to consider the exchanges that take place between each guest and the hotel's receptionist.

The main success scenario describes the simplest path for checking a guest into a room in a hotel. There can and will be exceptions to the main success scenario. Our solution is shown in Table 2.

Table 2 A textual description of the *check in guest* use case

Identifier and name	UC2 <i>check in guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest takes up a reservation and occupies a room at the desired hotel.
Precondition	There is a reservation for the guest for an available room of the desired type, and the guest can pay for the room.
Postcondition	The guest will have been allocated to a room for the period identified in the reservation, and a bill will have been opened for the duration of the stay.
Assumptions	The guest is already known to the hotel's software system. The hotel is confident that the guest can pay. For example, the guest has a valid credit card.
Main success scenario	
1	The guest provides a reservation number to the receptionist.
2	The receptionist enters the reservation number to find the reservation.
3	The hotel system provides the details of the requested reservation.
4	The receptionist confirms the details of the room type and duration of stay with the guest.
5	The hotel system allocates a room to the guest.
6	The hotel system opens a bill for the guest. (It could be that there is a separate billing package, which must be notified upon check in.)
7	The receptionist issues a key to the guest.

Even at this early stage in the development, it is possible to see that a further partitioning can be performed on the *check in guest* use case. The main business of hotels is to provide rooms for guests. In return, hotels require payment for this service. However, a typical hotel is likely to provide more than just the rental of rooms. For example, the hotel's restaurant could be available to the general public, and in larger

hotels there could be function rooms available for meetings and wedding receptions. As a result, we might decide to have a separate use case that deals with payment.

The main exceptions to the above scenario are related to a guest who does not have a reservation for some reason. We will explore ways of recording such exceptions in Section 4.

3.7 Summary of section

In this section, you saw how use case diagrams help you define what a proposed system should do from a user's perspective. Use case diagrams can also provide a basis for a contract between the customer and the developer because:

- ▶ use case models are used to drive the work to be done in software development;
- ▶ from a simple problem description, it is possible to draw a use case model that can be understood by people who are unfamiliar with UML;
- ▶ each use case should lead to a result that brings some observable value to at least one of its actors;
- ▶ the requirements of a use case can be captured through either a written statement or, more formally, the use of pre- and postconditions;
- ▶ detailed software requirements can be generated from each step of a scenario of a use case;
- ▶ a use case can be 'walked through' by one or more scenarios, which are particular sequences of possible actions within a given use case.

4

More details on use case models

In this section we will examine the relationships between use cases. You will also see two ways of refining a use case model to help you identify potential components and variations on existing tasks, when dealing with unusual circumstances.

On completion of this section you should be able to:

- ▶ identify a new use case that allows an original use case to deal with unusual circumstances;
- ▶ identify a subsidiary use case that provides functionality common to two or more use cases;
- ▶ describe the relationship between use cases and components;
- ▶ identify some of the problems associated with use cases.

4.1 More about actors

In the hotel example shown in Figure 9, the *Guest* actor is not associated with the use cases for checking in and out because we have assumed that the only real user for these two use cases is the receptionist; self check in and check out is not being considered.

An aim of the new system is to allow potential guests to use their web browser to make reservations, as an alternative to contacting a hotel directly. Hence the diagram shows *Guest* as an actor for the *make reservation* use case. However, since people will still be using other methods of requesting a room, such as telephone or letter, we should also allow for a member of the hotel's staff to perform the service, and so *Receptionist* is also shown as being associated with *make reservation*.

UML provides a notation to use generalisation between actors. You can change Figure 9 to show a new actor called *Reserver* associated with the *make reservation* use case. You can still show the actors *Receptionist* and *Guest* as specialisations of *Reserver*, as shown in Figure 11. By using the open-headed arrow from *Guest* to *Reserver*, you are saying that a *Guest* can do the same things that a *Reserver* can, but may do something else that a *Reserver* does not.

In larger hotels, a separate role may be needed for reservations. You would alter your use case model accordingly.

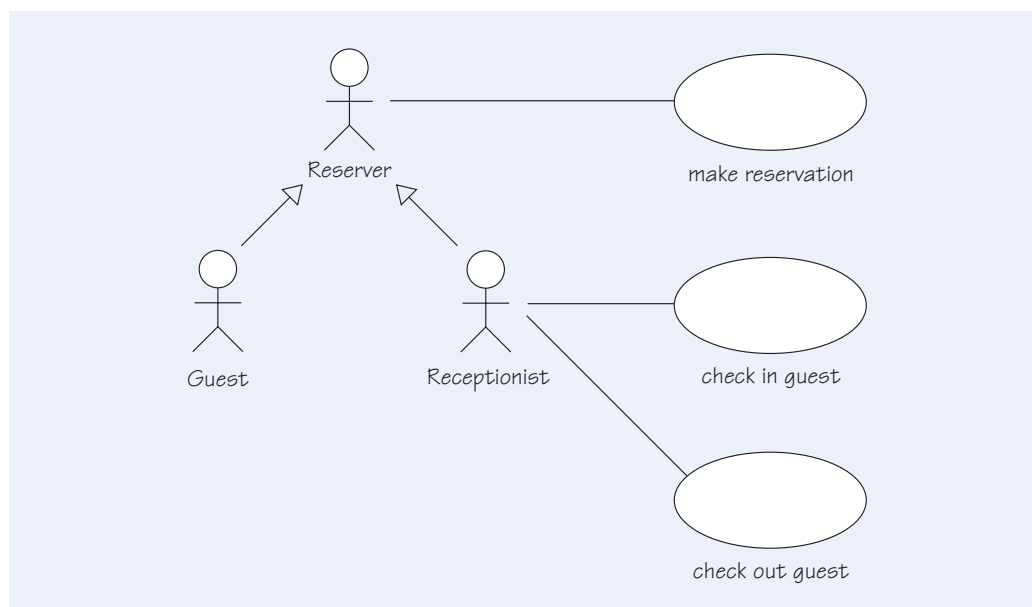


Figure 11 Relating the roles that actors play

SAQ 9

What is the purpose of identifying relationships between actors?

ANSWER.....

The purpose of identifying relationships between actors is to indicate generalisations. Figure 11 illustrates a *Guest* being a special type of *Reserver*. A *Guest* can do the same things that a *Reserver* can, but may be able to do something else that a *Reserver* cannot.

4.2 Modelling the relationships between use cases

Use cases can be related to one another. There are two very common and important forms of relationship.

- ▶ **Inclusion** This is when two or more use cases have an overlapping area of functionality – that is, a common task – that can be factored out as a distinct use case. The new use case can then be used by each of the original use cases, so avoiding duplication.
- ▶ **Extension** This is when a use case has a main success scenario but also alternative scenarios which demand a variation on the original use case – different or additional actions. Each variation can be separated out as a use case that is distinct from but related to the original use case. An extension is conditional while an inclusion is not.

In both situations, the new tasks become new use cases and, as you will see below, UML provides a suitable notation to represent the relationship between the original use cases and the new ones.

The main disadvantage of this approach is the additional complexity it brings to a model, in contrast to the simple use cases considered in the previous section. The simplest advice for you as the developer is to remember why you are creating the model and who it is for. We consider the problems of use case modelling later.

Stereotypes

UML is extensible; that is to say it is possible to add to its basic language. One way this can be done is by adding **stereotypes**. A stereotype can be used to specialise a UML model element of any kind. For example, a class can be given the stereotype «*abstract*», to show that it is a class of a special sort.

The convention is that a UML stereotype name is always enclosed between double angle brackets «...», called **guillemets**.

UML predefines a number of stereotypes, and we are at liberty to add our own. For instance, we could if we wished add a stereotype «*m363*», but we would need to agree on its meaning. Stereotypes that are most often used in practice are the ones already defined in UML, because they are generally accepted and understood. The two predefined stereotypes we are concerned with here are «*include*» and «*extend*».

Sharing behaviour between use cases – «include»

In the process of requirements elicitation and specification, you may find a certain amount of common behaviour in two or more of your use cases. You may even find that an existing software component can provide part or all of that common or shared behaviour.

You can record the shared behaviour in a new use case, and connect it to the use cases that it came from with a dashed arrow (indicating a dependency relationship) pointing from the original use case to the new one. Think of the new use case as always being included in each of the originals. Hence the dependency arrow is labelled with the «*include*» stereotype. Figure 12 shows an example taken from the domain of a lending library.

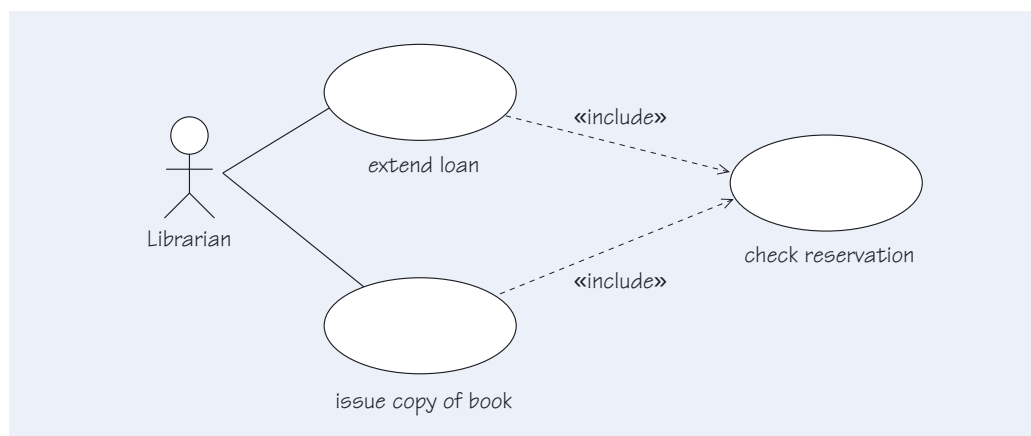


Figure 12 Shared behaviour in a lending library

Members of a lending library can borrow a certain number of books and, for any book borrowed, ask the librarian to extend the loan period. Before the librarian can issue a copy of a book to a member, there must be a check to make sure that there are no existing reservations for the book by other members. The same check needs to be carried out when a member presents a book to the librarian for renewal. The reservation check is a shared piece of behaviour, a common scenario, which can be developed separately as the *check reservation* use case. Note that this is *unconditional* behaviour – the reservation check must be performed whenever a loan is extended or a book is issued.

A dependency arrow has a *source* (where it comes from) and a *target* (where it goes to). In Figure 12, *check reservation* is the target use case for both dependency arrows; *extend loan* and *issue copy of book* are the source use cases. If the target use case

changes during development, the source use case will also be affected. In practice, the source use case will need to be reviewed if the scenarios in the target use case change. Note that the dependency is unidirectional. The target use case is not affected by changes in the source use case. (In the worst case, it simply becomes irrelevant and the dependency is 'broken'.)

The arrows for stereotyped associations point *in the direction indicated by the verb*. That is to say that, for «include», the arrow points to the use case that is included.

By taking out any common or shared behaviour, you can benefit from a simplification of the original use cases and make them easier to understand. There is a further benefit in terms of the internal consistency of the final requirements specification. Instead of having two or more different scenarios for a reservation check, for example, there will be just one main success scenario for the new *check reservation* use case shown in Figure 12.

In addition, there is a chance to consider the reuse of existing components and also the potential identification of new components. You need to keep in mind, as well, that a use case is only useful to the extent that it helps as a communication tool, and you should therefore only use «include» if it helps understanding of the model.

Alternatives to the main success scenario – «extend»

If a use case incorporates a scenario that is significantly different from the main success scenario, you may decide to create a new subsidiary use case. There may even be a need to create more than one subsidiary, depending on what happens in different circumstances. For example, a typical lending library will set an upper limit on the number of books that its members can borrow at any one time. A librarian will not be allowed to issue a copy of a book if that limit would be exceeded.

Figure 13 shows a fragment of a use case diagram that identifies a new use case, *refuse loan*. The new use case (the source) is connected to the original (the target) with a dashed arrow and labelled with the stereotype «extend». Note that the arrow points again *in the direction indicated by the verb*. For «extend», the arrow points to the use case that is extended. Beware! This one often catches people out, because they intuitively expect the arrow to point to the extension, but it does not: it goes the other way. The new *refuse loan* use case depends on the original. This is *conditional* behaviour, as it is only performed when the member would have too many books. (Compare this with the unconditional behaviour associated with an «include» dependency.)

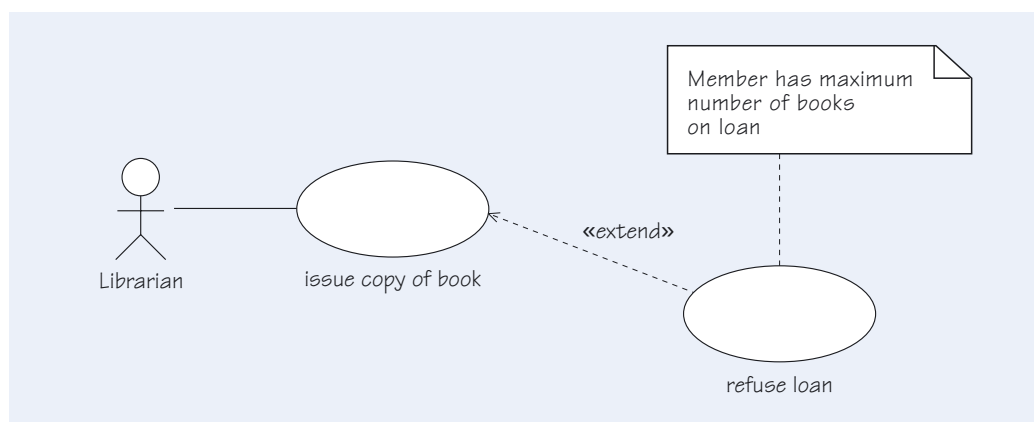


Figure 13 Alternative behaviour in a lending library

UML provides a number of ways to record the event that triggers the subsidiary use case. In Figure 13, we have used the general purpose notation for a note, to indicate that *refuse loan* is performed when a member already has the maximum number of books on loan.

The new use case should record a description of the corresponding scenario. There are two key points that it should contain:

- ▶ the condition that triggers the subsidiary use case;
- ▶ the places in the main success scenario where the condition is tested – called **extension points**.

Extensions can be treated more formally than a simple UML note on a use case diagram. Figure 14 shows an example taken from the description of the *make reservation* use case in the hotel-chain system. The *make reservation* use case has been modified to show the condition to be tested at an extension point in the main success scenario. The condition and the reference to the extension point are optionally shown in a note attached to the corresponding extend relationship (in UML 2).

Not all CASE tools support the representation of extension points inside a use case.

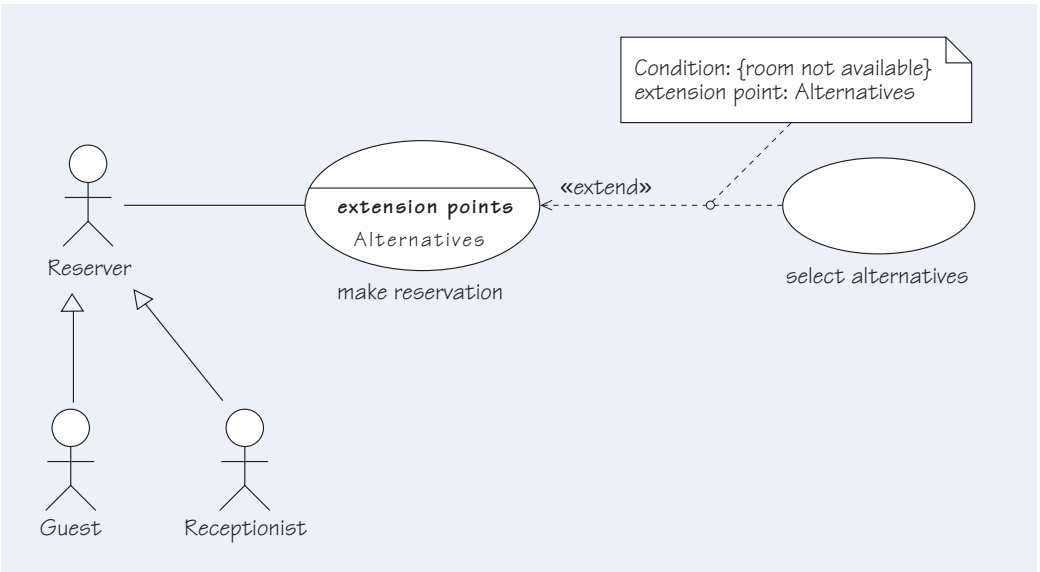


Figure 14 An extension to making a reservation

Table 3 shows how the original description of the *make reservation* use case contained in Table 1 has been changed to take into account the extension to deal with instances where the hotel has no free room available for the requested period, and the introduction of a new actor called *Reserver*. Each step in the main success scenario acts as a potential extension point, from which the relationship to a new use case can be defined. Here, step 3 is the extension point that leads to the additional steps described in 3.a and 3.b. As the extension point at step 5 shows, some work can be avoided if the subject of the reservation has stayed somewhere in the hotel chain before. Where such choices arise, your main success scenario should reflect the more dominant or typical flow. Table 3 reflects an emphasis upon new guests for the hotel chain.

Table 3 Extending the description of a use case in the hotel domain

Identifier and name		UC1 <i>make reservation</i>
Initiator		<i>Reserver</i> (may be a <i>Guest</i> or a <i>Receptionist</i>)
Goal		A room in a hotel is reserved for a guest.
Precondition		None
Postcondition		A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions		The expected initiator is a guest, using a web browser to perform the use case, or a receptionist. The guest is not already known to the hotel's software system (see step 5).
Main success scenario		
1	The reserver chooses to make a reservation on behalf of a potential guest.	
2	The reserver selects the desired hotel, dates and type of room.	
3	The hotel system provides the availability and price for the request. (An offer is made.)	
4	The reserver agrees to proceed with the offer.	
5	The reserver provides identification and contact details for the hotel system's records.	
6	The reserver provides payment details.	
7	The hotel system creates a reservation and gives it a number.	
8	The hotel system reveals the number to the reserver.	
9	The hotel system creates a confirmation of the reservation and sends it to the guest identified by the reserver.	
Extensions		
3.a	<i>room not available</i>	
3.a.1	The hotel system offers alternative dates and types of room.	
3.a.2	The reserver selects from the alternatives or declines the offer.	
5.a	<i>guest already on record</i>	
5.a.1	Resume at step 6.	

The convention we follow in numbering extensions is to use the number of the step followed by a letter; for example, extension 3.a starts an alternative path to step 3 in the scenario. If there is a series of steps in this extension they will be numbered 3.a.1, 3.a.2 and so on. If there are other alternative extensions for step 3 of the scenario, they will be numbered 3.b, 3.c and so on.

To extend or include?

Whatever kind of software system you intend to develop, there will be a need to consider its security. Usually, we allow only trustworthy people to use the new software system. Therefore, we can envisage a *log on* use case, which describes how a user

gains access through a unique identifier and a password. How should such a requirement be included in the example of the hotel chain, shown in Figure 9?

Analogously to natural languages, UML allows a number of ‘grammatically correct’ options each of which will make more or less sense depending on the context. For example, we could show the *log on* use case as a component of every use case that is associated with an actor, as shown in Figure 15.

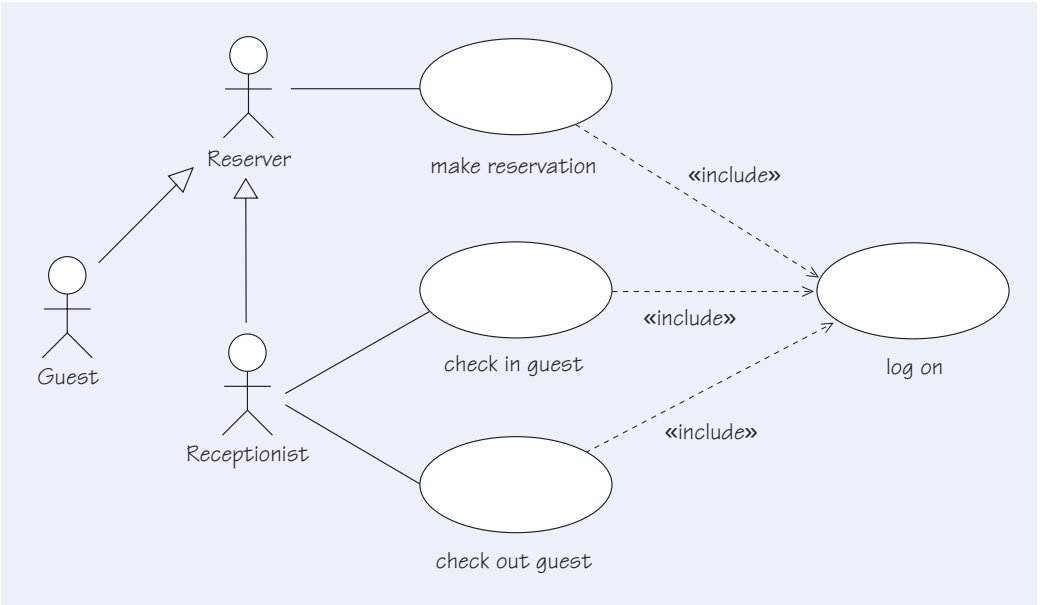


Figure 15 Including the *log on* use case in the hotel domain

You can also redraw Figure 15, as in Figure 16, showing the three original use cases as variations of the *log on* use case. It would be ‘grammatically correct’, although it would be difficult for the reader to see the intended purpose of the system, because we have shifted the emphasis to the action of logging on and away from the primary functions. So although this is grammatically correct in UML, it probably makes less sense than the first diagram. Note that although the use cases are placed differently in the two figures, there is no implication of ordering: time ordering is not shown in use case diagrams.

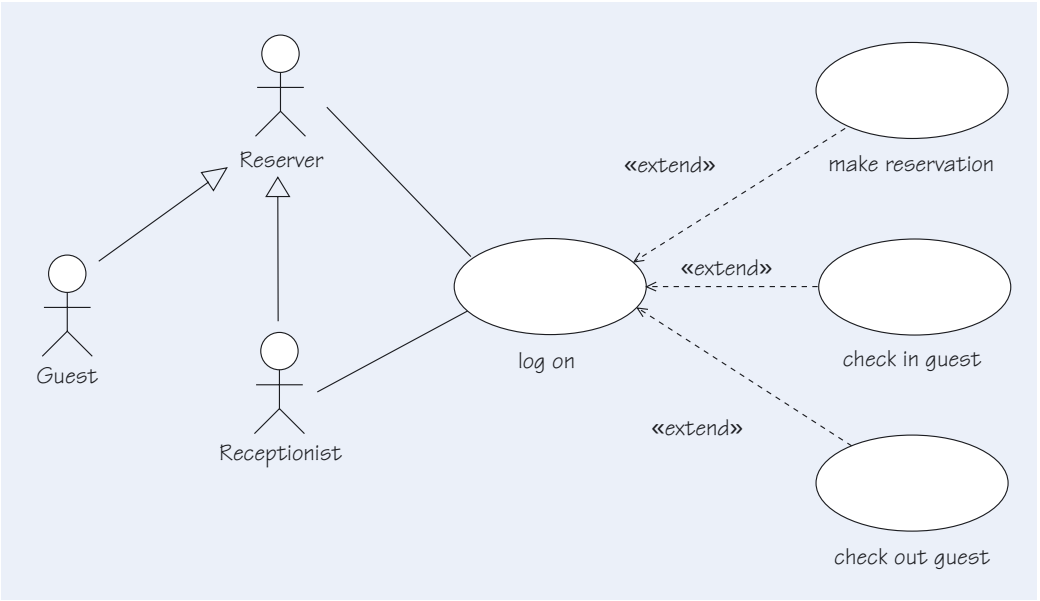


Figure 16 Making *log on* the main use case

Avoiding over-complex use case diagrams

You should beware of making your diagram too complex. The general approach to managing complexity is to partition a problem into subproblems, and use abstraction to reduce the detail and extract what is most significant to the problem. The same applies to use case diagrams.

Reduce the complexity of your use case diagram by:

- ▶ redrawing it at a higher level of abstraction;
- ▶ splitting it up into smaller modules that UML calls packages.

In the case of the hotel chain, we might partition our solution into three packages:

- ▶ reservations;
- ▶ checking guests in and out of their rooms;
- ▶ system access.

Each package may then be assigned to a separate developer for implementation. However, the project team must then deal with the dependencies between the three packages as they work towards a solution. We will return to the subject of packages in *Unit 7*.

SAQ 10

- (a) What is a stereotype in UML?
- (b) What is the function of the «include» stereotype?
- (c) What is the function of the «extend» stereotype?
- (d) Is it necessary to place the «include» and «extend» stereotypes on all diagrams?
- (e) How would you modify a use case model to show that you intend to employ a component that already exists?

ANSWER.....

- (a) A stereotype is a way of attaching extra classifications to a model. Stereotypes can be user-defined; this is a way of extending UML.
 - (b) The «include» stereotype indicates a situation where a use case is reused. In Figure 12, the diagram illustrates the *check reservation* use case, which is used by two other use cases. The purpose is to demonstrate commonality between tasks so that reuse can be achieved. The additional use case is included unconditionally in the original, or base, use case.
 - (c) The «extend» stereotype indicates a conditional extension to the original use case, known as alternative behaviour. This is used to illustrate a case where there are two or more significantly different scenarios, so that the main case and the additional, subsidiary cases are clearly differentiated. The main purpose of this classification is to separate out a special case. You should add a condition to each extension to specify when the variant behaviour will be included. This could be done with either a note or an extension point.
 - (d) No, it is not necessary to place the «include» stereotype and the «extend» stereotype on all diagrams. In fact, in some situations they can cause confusion since they will not be understood by everyone.
 - (e) Each use case that benefits from the component must have a relationship to that component shown on the diagram. This relationship should have the «include» stereotype attached to it.
-

4.3 Problems with use cases

The above example about access to the hotel system illustrates a more general modelling problem for the developer. It is often difficult to separate a problem from its solution. In practical terms, you might express it as a question about the activities of analysis and design. 'Am I analysing the problem or designing a solution?'

In object-oriented software development, the question is hard to answer. You may find it easier to think of analysis as the activity of investigating a problem and opening up choices, in contrast with design as the activity of taking decisions and narrowing down the number of choices to arrive at a solution. In practice, a use case diagram represents a vision of what the solution, a new software system, will do for its users.

With use cases, you also run the risk of straying into a top-down, functional decomposition and away from the object-oriented viewpoint that is embedded within UML. It is easy to decompose each use case into smaller use cases in your search for reuse through the «include» stereotype. Indeed, if you are making your project plans according to the use cases that you identify, the urge to find a use case of a size that you can easily estimate is understandable. A good project manager will make some assessment of this risk and review it upon each iteration of the life cycle.

Another common problem with use case modelling is deciding the size and scope of each use case. There is no consensus on this issue, because of the wide variety of contexts and viewpoints. However, we can say that a use case should be larger than a single method of a class but smaller than a business process. In the hotel chain, for example, the handling of reservations would be treated as a business process separate from checking in and out. That is to say, *make reservation* is only one of the tasks in the process of handling reservations.

No single technique can guarantee that you will collect and identify all the users' requirements. So, if you spend too much time modelling use cases, you can become distracted by the process of modelling and lose track of the main aim, which is to capture the behavioural requirements for a software system. Consequently, you should use more than one technique to produce a requirements specification.

We may set off to produce a use case diagram which is intelligible to customers, but allow it to get so elaborate that they can no longer understand it. Having decided to represent requirements in one or more use case models, the most important thing to consider is their intended audience. You need use cases that can be read and understood by the domain experts as well as the team of developers. The domain experts usually come from the customer's area. If you cannot demonstrate the benefits of your proposed solution to them, there is little chance that it will be acceptable to the customer.

In the same way, use case models must be useful to the rest of your development team. For example, those who will be testing the software system must be able to generate their test scripts from the use cases and the subsequent design artefacts. In the approach followed in the course, by detailing software requirements for each step of a use case, and defining a fit criterion for each of these requirements, the path is well defined for the development team.

In terms of the content of each use case diagram, you should avoid the use of the «include» and «extend» stereotypes for an audience that is less familiar with UML than your team members. The simple notation for actors and their associations with use cases has been a factor in their favour.

The problem of when to stop modelling of any kind is affected by a number of factors that are beyond the scope of this section.

SAQ 11

- (a) What is the main source of confusion for a developer who has been asked to produce a use case diagram for a potential software system?
- (b) What problems may arise when developing a software system from a set of use cases?

ANSWER.....

- (a) The main source of confusion for the developer is the separation of the problem from its potential solution. In asking ‘What tasks should the software system perform?’, for example, the developer should be aiming to draw a clear boundary between the problem domain and the intended software system.
- (b) One problem is that the focus may end up being top-down and function-oriented, resulting in an inflexible and difficult-to-maintain system. Focusing on the use cases may cause the developers to sacrifice the object-oriented nature of the system, thus losing any advantage that UML offers.

Another danger lies in mistaking design for requirements, where a design decision is mistaken for a constraint. Focusing on the requirements in a use case may cause developers to view the system too operationally, where a sequence of events is assumed to be the only answer. Developers need to distinguish between requirements and preferred designs.

There is also a danger of missing some of the requirements, especially if emphasis is placed on actors and their tasks, because not all the requirements will emerge in this process. A developer should use more than just one model of a proposed system. For example, class modelling (discussed in Unit 5) should be performed alongside use case modelling, since one informs the other. This illustrates that use cases help mainly with requirements capture and testing but not with the design.

Use cases need to be used in a way that is understandable to the customer but also useful to the developers.



Exercise 7

What are the tasks involved in preparing a use case diagram for a moderately large system, intended for the development team?

Solution.....

For each use case diagram:

- ▶ define the context for the model by identifying the actors involved in the aspect of the system in question;
- ▶ analyse the behaviour that each actor expects from the proposed system, and identify the use cases (as units of functionality within the overall requirements);
- ▶ identify the common behaviour that may be reused by other actors, and the variations on common behaviour (the stereotypes «include» and «extend»);
- ▶ draw a model that shows the use cases, the actors and the relationships between them;
- ▶ annotate the use cases as you learn more about the requirements.

For large projects, you will need to record separately any constraints that affect more than one use case diagram. One way is to produce a use case model at a higher level of abstraction.

Exercise 8



Recall the hotel example, which was introduced in Section 3 (see Figures 8 and 9). Redraw Figure 8, taking into account the information contained in Tables 1, 2 and 3, to show common tasks and any extensions to the main success scenarios.

(Do not show the *log on* use case, described in Figure 15.)

Solution

Our solution is given in Figure 17.

When a *Guest* checks in, we can identify two components: *identify reservation* and *check for available rooms*.

When a *Guest* checks out, we can identify at least one component: *prepare bill*. A bill is assumed to exist because the *check in guest* use case includes *open new bill* for the guest. This is something for the developer to check, in discussions with the users. We will also assume that the records of payments made by guests are held separately, which is a typical accounting procedure. Hence we have identified a new actor called *Accounts System* that communicates with *open new bill* and *prepare bill*.

Since a room must be cleaned after a guest has checked out we have identified a use case named *request room cleaning*. This is the kind of detail that you are likely to discover as you find out more about the problem domain.

Finally, there may be an early design decision to change the way that reservations are created. The difficulty arises because there is a subsidiary goal to create a new guest record if the person identified during reservation is not known to the hotel system. For example, you may decide to have a software component that identifies those who have already been entered into the hotel system (in a use case called *identify guest*, say). Such a component is very likely to be reused in other use cases. As a consequence, creating a new guest could be seen as an extension of *identify guest*, whenever a condition such as *guest not found* is encountered.

Undoubtedly, your diagram will differ in some aspects from Figure 17, but you should be able to justify your choices. You should have used «*extend*» to show variant behaviour, and «*include*» to show the potential use of components. We have deliberately added enough new use cases to show how easy it is for use case diagrams to become quite complicated. Such complex use case diagrams are unlikely to be shown to the intended users; they are intended for the development team.

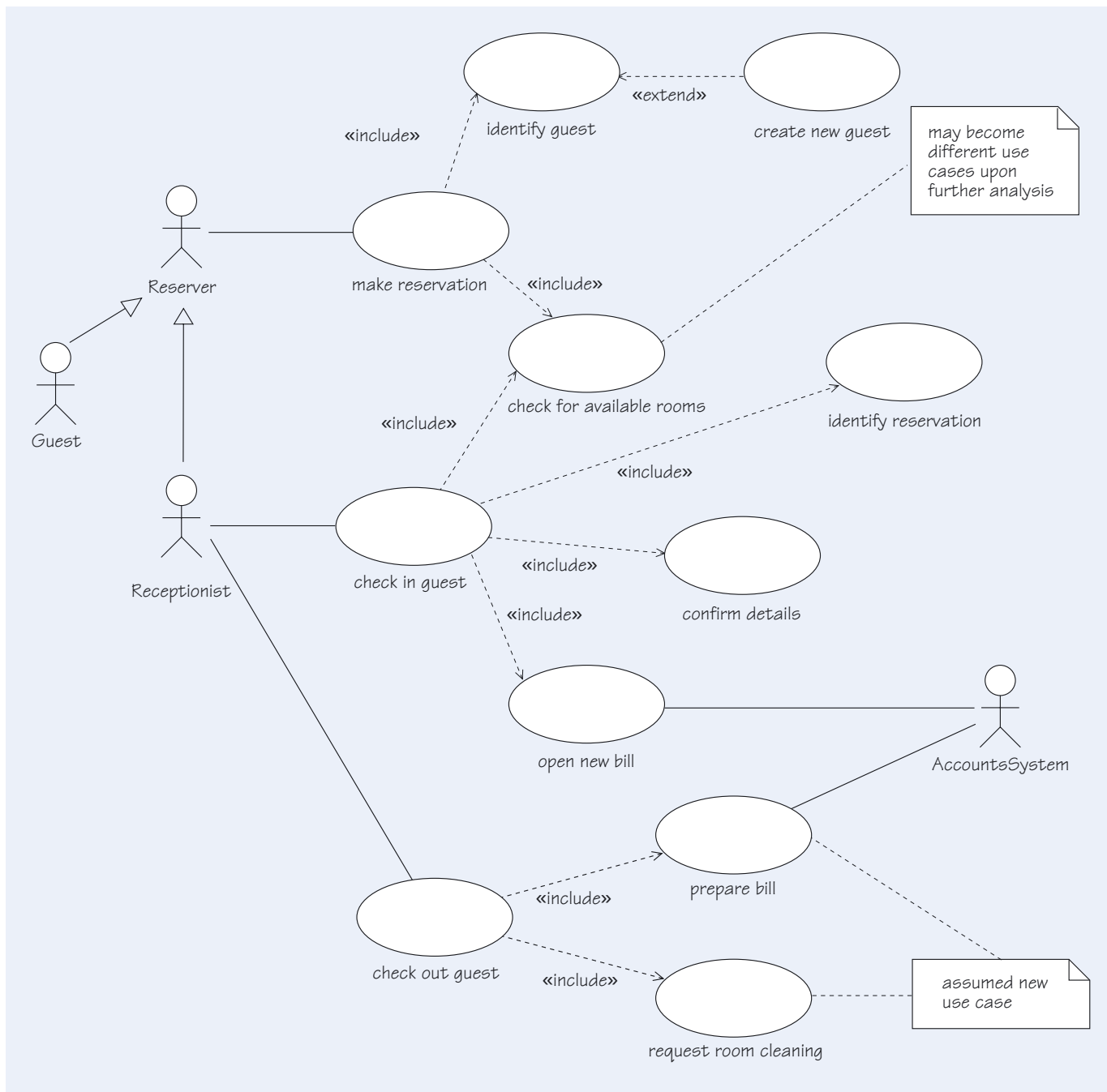


Figure 17 A use case diagram for a hotel



Exercise 9

A nationwide chain of DVD rental shops wants to take advantage of web technology to change the way that it operates. The main change is to allow members to use their web browsers to look through its stock titles, make reservations and rent DVDs for delivery to any location within a certain distance from any one of its shops.

Currently, prospective members have to go in person to their local branch of the DVD chain and pay a joining fee. A member of staff at the branch then issues a membership card, which is used to reserve or rent DVDs. Members can only rent from the stock available at the branch where they joined.

The owners want to simplify the membership procedure as follows. People can use a web browser to join the scheme by providing credit card details as well as their postcodes, which allows the company to perform a credit check and locate their home address while the prospective member waits. Once a membership application has been accepted, the member can browse the stock of DVDs and find out when their choice will be available for rental from the branch nearest to the member's home.

The new software system should enable a member to reserve a DVD in advance or request its delivery if it is available. The local branch would deliver it to the requested address for a small charge. The normal DVD loan period is 48 hours. The member would either return the DVD to the branch in person or request its collection for an additional charge – a service that must be asked for when the rental request is made.

The customer also wants the software system to enable members to request an extension to the loan period (by a further 48 hours) after paying an extra charge, but they must be prevented from doing so if they have any DVDs that are overdue, or if the DVD has been reserved by another member.

Draw a use case diagram for the proposed system, and identify the constraints or assumptions that you make.

Solution

The notion of renting DVDs is very similar to that of lending books, which you saw in Example 1. That is, the two domains are similar, which should help you identify the actors. (We need to take care, of course, that we do not make unjustified assumptions.)

A good first step is to establish the context for the proposed system by identifying the actors that surround it. When you have identified the actors, you can investigate the behaviour that each one will expect from the proposed system, which will eventually form the use cases for your model.

Working with the analogy to the library case study, the two basic actors are:

- ▶ *Member*
- ▶ *DVDLibrarian*

The new scheme also suggests that we should identify an actor who delivers and collects DVDs. For the present, we will call this actor a *Driver* until we can find out more about the requirements. A less obvious actor is the person who wants to become a new member. From the description, we find that potential members must have a credit card, which is a constraint restricting the kind of people who can join. This raises the need for a connection to an external system that can perform credit checks. Thus there are two more actors:

- ▶ *CreditCardHolder*
- ▶ *CreditAgency*

The library example helps us find a number of appropriate use cases for the DVD chain:

- ▶ *issue copy of DVD*
- ▶ *return copy of DVD*
- ▶ *reserve DVD*
- ▶ *update catalogue*
- ▶ *browse stock*

There is an additional use case that is shared by more than one actor, which we will call *join scheme*, since it links credit card holders with the *CreditAgency* authorisation service.

We are not well informed about the delivery and collection service of the new scheme, and so that is an area for investigation during analysis. For the moment, we will make the assumption that a driver can carry out two use cases. The two are separated to allow the possibility of making additional charges for the collection of DVDs in the new scheme:

- ▶ *identify DVDs for delivery*
- ▶ *identify DVDs for pick-up*

There are three use cases involved in extending loans:

- ▶ the main use case for members to extend their loan;
- ▶ a variation that refuses loans when a member has items that are overdue;
- ▶ one that checks for existing reservations before allowing a DVD to be rented or reserved.

The *check reservation* appears to be a key component in the proposed software system, since it is associated with four other use cases. You might consider its early implementation within the project.

The question only mentioned the refusal of a loan in the case where a member tried to extend a rental. As a developer, it is worth noting that such a requirement may spread to other use cases such as *issue copy of DVD* and *reserve DVD* – a point for further analysis. Figure 18 includes such an extension for the *issue copy of DVD* use case.

When each copy is returned to the branch, it is possible that there will be some outstanding charges, especially if members are late in returning their DVDs. The *collect charges* use case may also have to deal with inconsistent records, which may occur when members keep extending the loan on a particular DVD.

The diagram in Figure 18 is one possible use case diagram for the proposed system. You may have used slightly different names on your diagram, and you may have identified alternative use cases to those we have shown. There is no one correct solution; there are many possibilities. You might have tried to share some of the use cases, but this would assume knowledge of the internal structure of the proposed system. You should check that in your use case diagram you have included the behaviour that will bring some discernible value to the actors. For example, as the driver's job is to ferry DVDs between members' homes and the local shop, he or she will derive benefit from the computer system by being provided with lists of addresses and the names of DVDs to be fetched or delivered.

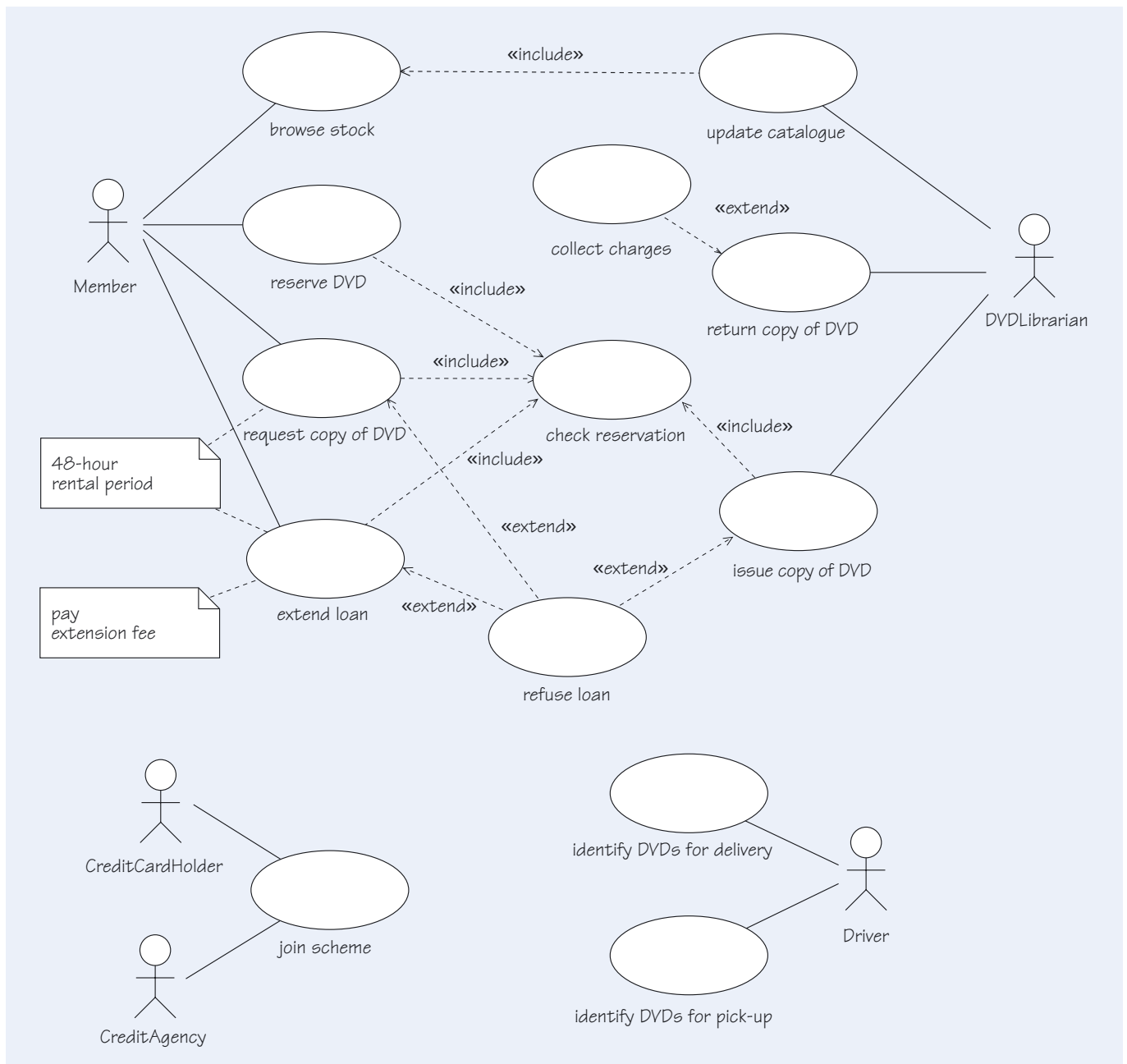


Figure 18 A DVD rental business

Finally, you should have included some notes with both the diagram and the elements in it. For example, the diagram shows a note with the *request copy of DVD* use case indicating the loan period. This is an excellent way to record constraints on the requirements.

4.4 Summary of section

In this section, you have seen that use case models play a key role in the development of a software system, especially when the user is placed at the centre of a development process. Use cases help you record the proposed requirements for a software system and add detail to their relationships as follows.

- ▶ The «extend» relationship between a new use case and an original use case shows how the system should deal with conditional behaviour.

- ▶ The «*include*» relationship shows how a subsidiary use case can provide some common functionality to two or more use cases at a higher level of abstraction.
- ▶ The «*include*» relationship between use cases shows how the system can employ a pre-existing component.

Hence use cases provide a starting point for project and quality management.

5

Getting users involved

In this section you will learn how to involve the users in the process of software development.

On completion of this section you should be able to:

- ▶ describe the connection between prototyping and the design of the user interface;
- ▶ identify the users' activities that may require an interface to the proposed software system.

Requirements capture involves identifying, recording and documenting the things that the intended users do as part of their interaction with a system. Usually, a developer will produce textual descriptions of the potential scenarios and attach them to the relevant use cases. In Section 2, you saw that an activity diagram enables a developer to model one or more scenarios in order to understand the work involved in the users' domain, such as a library.

However, the developer is an expert in a particular domain, which is software development, whereas the users are experts in their own domain. Even with the support of experts in the users' domain, the most significant mistakes are usually made during analysis. The cost of rectifying such errors increases the later they are detected in the development process. Thus identifying use cases, and then testing them, helps to detect or avoid the kinds of error that might arise:

- ▶ by missing a requirement;
- ▶ by not understanding a requirement;
- ▶ through ambiguity.

One of the ways to improve the analysis and identification of requirements is for the developers to produce a prototype. This was first discussed in Subsection 3.2 of *Unit 1*.

5.1 Prototyping for and with the user

One of the main uses of prototyping is to help with the design of the user interface for the proposed software system. It is a direct way of getting the users involved with the development of the software. By working closely with the users, you can minimise or avoid any misunderstanding and consequent false expectations on the part of either the developer or the users. So when the final product becomes available, it is less of a surprise to its users, and this helps the customer overcome any resistance to change that can undermine the investment made in the software system.

The activity diagram in Figure 19 illustrates the general process of prototyping. There are two important guidelines that should be followed. First, the developer and the user must know what the particular prototype is to be used for, and they need to agree on a goal that must be achieved. This will include an agreed schedule for building the prototype.

Second, the prototype should be evaluated against its goal.

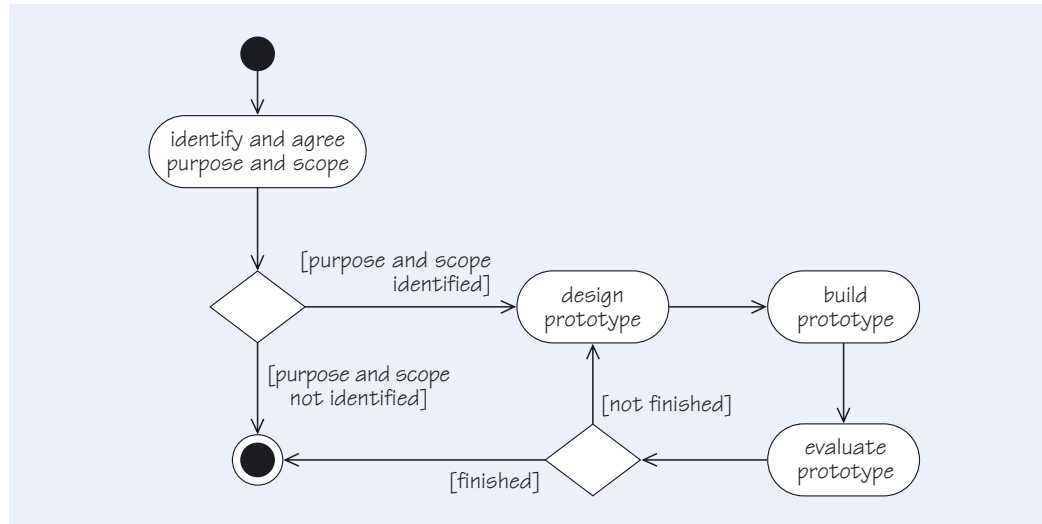


Figure 19 An activity diagram for the prototyping process

From the developer's point of view, it is important not to agree to produce a prototype that does not benefit the users.

A prototype is not intended to be a complete working version of the software. It is not necessary to spend a great deal of time on getting the code 'right', because you may choose to scrap the code after completing the evaluation of the prototype. For this reason, a prototyping tool can be beneficial. For example, a good tool can help you assemble screen or report layouts quickly and rework them after each evaluation.

Note that the main goal is to make sure that the prototype meets the needs of its intended users. It is time to stop when you are not generating any new requirements (or when there are only minor requirements that would not be paid for by the customer). Next, the developer needs to document what has been achieved. This will provide the information on the final software system for all those people involved in its development.

Finally, the people who should be involved in the development of the prototype are the 'real' users – those who will use the interfaces that are developed from the prototype. They know their own needs for their day-to-day role, and they have the most to gain from a successful implementation.

SAQ 12

- (a) Who initiates the prototyping process?
- (b) Who should test a prototype?

ANSWER.....

- (a) The developers would normally start the prototyping process because they have detected or identified a particular problem in their analysis of the requirements for the proposed system. (But note that the project manager would need to get approval from the customer for the additional effort.)
- (b) The intended users should test it. For example, if you developed a series of interfaces as part of a prototype for the borrowing and returning of books, the librarians would be the testers.

5.2

Linking activities to user interfaces

Before any prototyping of interfaces can take place, the developer needs to identify the appropriate interfaces from the users' requirements and determine where and when they will be needed. One way to achieve this goal is to use an activity diagram.

Consider the borrowing and returning of books that you saw in Example 1, which we associate with the following two use cases:

- ▶ *issue copy of book*
- ▶ *return copy of book*

Figure 3 shows the activities involved when a member borrows a copy of a book. It shows a transaction that takes place between the member and the librarian. The member has taken the copy of a book from a shelf. The librarian needs to record the fact that the member has borrowed a particular copy of a book. So the librarian needs an interface with the software system to record the transaction. This interface will be needed when the member is at the front of the queue and the librarian is ready to attend to that member.

The transaction is complete when the librarian has:

- ▶ identified the member who wants to borrow the book;
- ▶ recorded the change in location – the book is with a particular member and is no longer on a library shelf;
- ▶ recorded the date when the book is due back in the library;
- ▶ told the member the date when the book should be returned to the library.

Thus the user interface for the use case *issue copy of book* must be available to the librarian when the librarian reaches the *record loan* activity between the two synchronisation bars in Figure 3.

Figure 4 shows the transaction for returning the book, which requires another user interface for the developer to consider. Hence the developer can record the associations between activities and interfaces, as Table 4 shows.

Table 4 Associating interfaces with user activities

Activity	Interface
<i>record loan</i>	<i>issue copy of book</i>
<i>record return</i>	<i>return copy of book</i>

Although we have identified two user interfaces for further analysis, we must also consider the consequences for the other use cases within the library such as:

- ▶ *reserve book*
- ▶ *extend loan*
- ▶ *browse catalogue*
- ▶ *update catalogue*

Consider the following description, which contains a number of design constraints on the developer.

There will only be one librarian available to staff the issue desk at any one time. So all members have to join a single queue. We use a bar code reader to identify items in our catalogue and individual members, who must present their membership card when they borrow books.

The developer can also gain a greater understanding of the domain through the prototyping process.

Suppose that the first iteration of the software system is concerned with the borrowing and returning of books, which sets the scope for a possible prototype. The developer should focus on a prototype of the user interface that will allow the librarian to deal with the two use cases *issue copy of book* and *return copy of book*.

One of the goals for a prototype would be to construct an interface for the librarian, which would allow the efficient *recording* of loans and returns. For instance, the developer of a prototype would try to optimise the activities *record loan* and *record return* for books. In return, the librarian would test the prototype to find out what is good or bad about it. Both librarian and developer should ensure that nothing is 'missing'. The librarian can see whether or not each requirement has been addressed, thereby relating requirements to implementation (that is, there is traceability in the design of the prototype).

Finally, we must stress that software development involves a variety of modelling activities, which we will continue to investigate in the following units. For example, we must look for the real-world objects that go into our conceptual model, which means that we must also look at a class model.

Use case and activity diagrams are only a starting point. They are an incomplete representation of a software system, and you may have to return to them for modification in a later iteration of the project.

SAQ 13

- What is the main benefit of identifying the location and availability of user interfaces in your activity diagrams?
- What does the librarian record in the transaction for the *return copy of book* use case, which Figure 4 illustrates? Can another member borrow the returned book?
- Suppose the developer built a prototype interface that could deal with all four use cases in the first iteration of the library system. Suggest a criterion that a librarian might use when evaluating the prototype, which goes beyond the completion of the underlying tasks within each use case.

ANSWER.....

- The main benefit of recording user (or any other) interfaces in an activity diagram is traceability. To the users, the interface *is* the software system: an unacceptable interface can lead to failure. The user interface is the link between what the users want and what the developers produce in response.

Also, the developer can identify the relative importance of each user interface for the project plan, particularly when resources are needed for a prototype.

- The librarian records that the book has been returned. Although the book is now back in the library, it does not automatically follow that another member can borrow it immediately. The developers will need to explore this detail with the librarians, who understand the business processes of a library in more detail.

- This was the example we thought of, but you may have thought of others.

Suppose there is the design constraint that at any given time only a single librarian will be on duty. Then that person will be faced with a queue of library members who may require a range of different services.

Librarians will find it essential that in these circumstances the system makes it very easy to switch between different tasks, to meet the varying types of request received. They will expect the prototype to provide evidence that the eventual system will meet this criterion.

Exercise 10



Draw an activity diagram to show the separate responsibilities of the receptionist and the hotel system for the *check in guest* use case. Your diagram should contain the main success scenario. You do not need to show any extensions. (See Figure 7 of this unit.)

Solution

We can use the textual description of the *check in guest* use case from Exercise 6 of this unit. Figure 20 shows an activity diagram showing the exchange of information that takes place at the user interface.

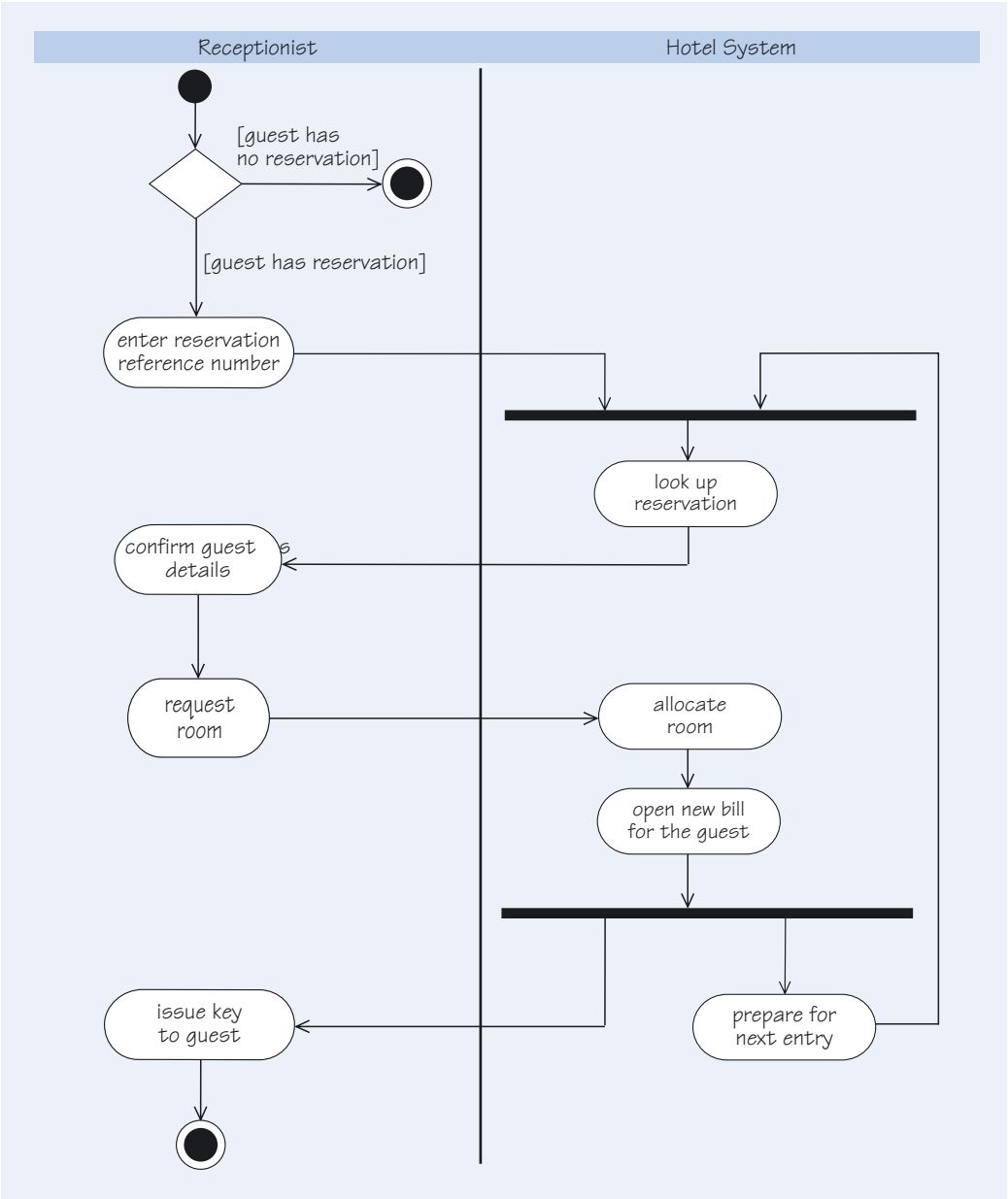


Figure 20 Separating the responsibilities during a prototyping activity

The receptionist initiates the use case, as shown in the diagram. Some of the activities show the conscious acts that a receptionist needs to undertake in order to make progress in the main success scenario for checking in a guest.



Exercise 11

Recall the requirements for the nationwide chain of DVD rental shops in Exercise 9. A member can reserve a DVD in advance or request its rental directly, if it is available. In addition, members can extend the period of loan by paying an extra charge, but they must be prevented from doing this if they have any DVDs overdue.

- (a) Draw an activity diagram for the proposed system to deal with the extension of a loan period. For simplicity, show only the main scenario, and do not use swimlanes for the different actors in your activity diagram.
- (b) When answering part (a), you will have come across a range of issues that the existing requirements do not cover. These would have to be clarified in discussions with the clients. List some of the issues which you can identify as requiring clarification, and state what points would need to be resolved in each case.

Solution

- (a) The activity diagram in Figure 21 shows one of the ways of extending the loan period for a DVD. We have shown the basic scenario without adding to the complexity of the activity diagram by looking for the exceptions that can and will arise. Discussions of the *extend loan* use case with the users should help identify the exceptions and their consequences.

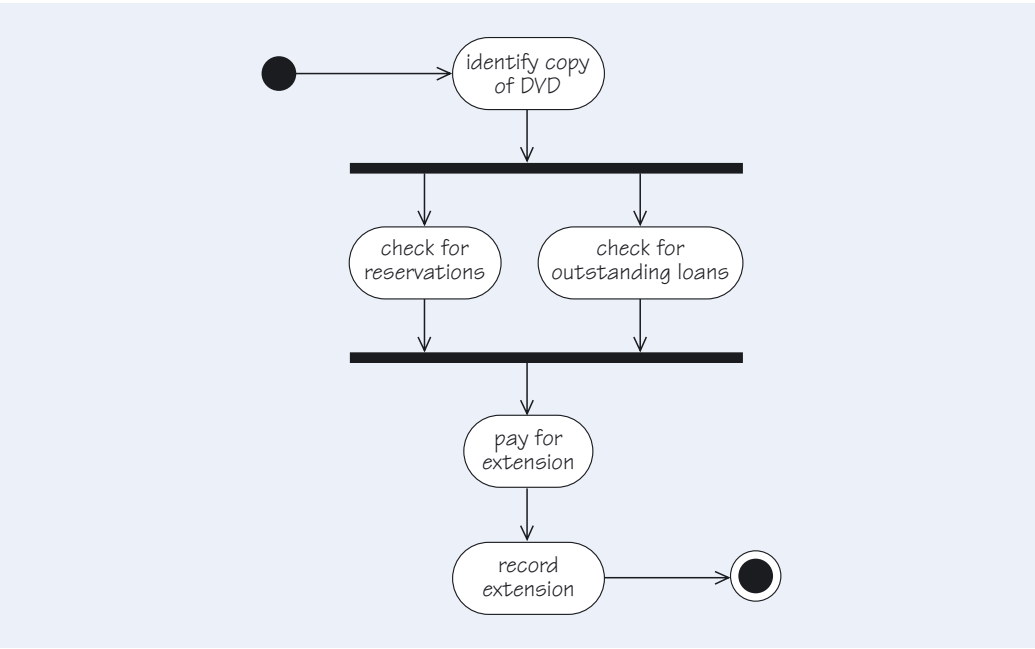


Figure 21 Extending the loan of a DVD

- (b) From the descriptions in Exercise 9, we are told that members use their credit cards for joining the scheme. As a developer, you would need to clarify the rules for other payments, such as rental charges and the charge for collection by a driver. So a discussion of the *extend loan* use case with the user might help you understand how the owners of the shops want to collect the charges. For example, is there a request to the *CreditAgency* actor for each payment, or would you allow certain members to accrue charges in an account?

Another point for discussion with the user involves the *refuse loan* use case, which is associated with the main *extend loan* use case. It may become a potential component for other use cases since it is a reasonable assumption that new transactions would be refused when a member has outstanding loans.

Another question relates to the constraint on the charge for collection. In the original requirements of Exercise 9, this fee had to be paid with the rental. The user

could consider whether or not to allow a new collection charge to be included as part of the *extend loan* use case. Figure 21 shows that the extension to the loan is made on the assumption that fees are paid.

The next step should be to consider the relationships between use cases. For example, the initial activity to identify the particular copy of a DVD could also be a component in the *browse stock* use case. Similarly, the *record extension* activity shares functionality with the *request copy of DVD* use case.

As part of your analysis, consider which of the activities would be 'visible' to the user and which ones would be 'hidden'. For instance, both the checking activities may be hidden and only become visible when there are any exceptions such as other members waiting for a copy of the DVD.

5.3 Summary of section

In this section, you have seen that activity modelling can help identify when the user would need to use an interface to the proposed software system. You can use activity diagrams for any of the scenarios of a use case where some interaction with an actor is expected. Prototyping the user interface brings developers and users into close contact with each other. By working together it is possible to minimise any misunderstanding that might lead to false expectations on either side.

6

Summary

In this unit, you have learnt how modelling can help with the process of requirements engineering in a variety of situations. In particular, you studied the roles of activity diagrams and use cases.

You should know how to draw simple use case diagrams that model the main tasks, the shared behaviour between those tasks, and the exceptions to the main success scenario. You have explored how use case and activity modelling can be used to help in consultations with the users, and have also seen some of the limitations of these techniques. You have seen how activity modelling can help the developer identify when users would need to have an interface available in order to complete their activities in different scenarios, and the role that prototyping plays in this task.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ use techniques such as activity diagrams that allow you to understand a business domain and model the context of a proposed software system;
- ▶ use modelling techniques such as use cases to help with the elicitation and detailed representation of requirements;
- ▶ apply use cases confidently and discuss their use throughout software development;
- ▶ carry out a detailed analysis of users' activities;
- ▶ discuss the connection between users' activities and the interface to a software system.

References

Eriksson, H. E. and Penker, M. (2001) *Business Modeling with UML: Business Patterns at Work*, New York, Wiley.

Krutchen, P. (2000) *The Rational Unified Process: An Introduction* (2nd edn), Reading, MA, Addison-Wesley.

Index

A

- activity diagrams 8
 - activities 8
 - decision nodes 10
 - forks 9
 - guards 10
 - joins 9
 - merge nodes 11
 - start 9
 - stop 9
 - swimlanes 11
 - synchronisation bars 9
 - transitions 9

- actors 19

- alternative behaviour 38

- associations 20

B

- business objects 21

- business processes 8

- business rules 6

C

- coherent units of functionality 20

D

- decision nodes 10

E

- extend, relationship 38

- extension 36

- extension points 39

F

- forks 9

G

- goals 5

- guards 10

- guillemets 37

H

- human–computer interaction 21

I

- include, relationship 37

- Inclusion 36

J

- joins 9

M

- merge nodes 11

N

- non-functional requirements 21

- notes 39

P

- postconditions 25

- preconditions 24

- processes 8

- prototyping 51

R

- resources 5

- reusing components 38

- rules 6

S

- scenarios 25

- shared behaviour 37

- stereotypes 37

- swimlanes 11

- synchronisation bars 9

- system boundaries 22

T

- tasks 20

- transitions 9

U

- use case diagrams 20

- use cases 19

- user-centred design 21

V

- validation of business rules 7

- verification of business rules 7

W

- walk-throughs 29

- workflows 11